DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **WEB TECHNOLOGY** Subject Code: **CS T63**

UNIT-III

XML: Introduction- Revolutions of XML-XML Basics – Defining XML Documents: DTD-XML Schema-Namespaces – XFiles: XLink – XPointer - XPath - XML with XSL – XSL-FO-Parsing XML using DOM-SAX-Integrating XML with database – Formatting XML on the web.

2 MARKS

1. **What is meant by XML?**
   - XML stands for eXtensible Markup Language
   - XML is a markup language much like HTML
   - XML was designed to store and transport data
   - XML was designed to be self-descriptive
   - XML is a W3C Recommendation

2. **What are the uses of XML?**
   - E-Business and E-Commerce
   - Content Management
   - Web Services and Distributed Computing
   - Peer-to-Peer Networking and Instant Messaging

3. **What are XML delimiter characters?**

Character	Meaning
<	The start of an XML markup tag
>	> The end of an)> less than angle brackets> XML markup tag
&	The start of an XML entity
;	The end of an XML entity

4. **What are the major portions of XML Document structure?** The major portions of an XML document include the following:
   - ✓ The XML declaration
   - ✓ The Document Type Declaration
   - ✓ The element data
   - ✓ The attribute data
   - ✓ The character data or XML content

5. **Write the XML Syntax Rules?**
   - All XML Elements Must Have a Closing Tag
   - XML Tags are Case Sensitive
   - XML Elements Must be Properly Nested
   - XML Documents Must Have a Root Element
   - XML Attribute Values Must be Quoted
   - Entity References
   - Attributes may only appear Once in the Same Start Tag
   - Attribute Values Cannot Contain References to External Entities

6. What are the components of the XML declaration?

Component	Description
<?xml	Starts the beginning of the processing instruction
Version="xxx"	Describes the specific version of XML being used in the document
standalone="xxx"	This standalone option defines whether documents are allowed to contain external markup declarations. This option can be set to "yes" or "no".
encoding="xxx"	Indicates the character encoding that the document uses. The default is "US-ASCII" but can be set to any value that XML processors recognize and can support. The most common alternate setting is "UTF-8".

7. What is DTD? **(NOV 2017)**

- The purpose of a **DTD (Document Type Definition)** is to define the legal building blocks of an XML document.
- A DTD defines the document structure with a list of legal elements and attributes.
- It enables an XML parser to verify whether an XML document is valid.
- DTD expresses the set of rules for document structure using an EBNF (Extended Backus-Naur Form) grammar.

8. What is the use XML element?

An XML document contains XML Elements.

An XML element is everything from (including) the element's start tag to (including) the element's end tag. An element can contain:

- ✓ text
- ✓ attributes
- ✓ other elements
- ✓ or a mix of the above

9. What are XML naming rules?

The elements must follow these naming rules:

- Element names are case-sensitive
- Element names must start with a letter or underscore
- Element names cannot start with the letters xml (or XML, or Xml, etc)
- Element names can contain letters, digits, hyphens, underscores, and periods
- Element names cannot contain spaces

10. What is the use XML attributes?

- XML elements can have attributes in the start tag, just like HTML.
- Attributes are used to provide additional information about elements.
- Attribute values must always be quoted.
- Either single or double quotes can be used

11. What are the advantages of XML?

- XML was designed to transport and store data
- XML tags are not predefined. You must define your own tags
- XML is designed to be self-descriptive
- With XML You Invent Your Own Tags
- XML is everywhere

12. Write the disadvantages of using XML attributes? **(APR 2017)**

- attributes cannot contain multiple values (but elements can)
- attributes are not easily expandable (for future changes)
- attributes cannot describe structures (child elements can)
- attributes are more difficult to manipulate by program code
- Attributes are difficult to read and maintain
- Use elements for data.

13. What are xml parsing errors?

Each parsing software can handle different errors has different parsing error messages. The common errors are:

- closing tags
- elements not matching up
- closing quotations
- Incorrect order

14. What is Processing Instruction?

Processing instructions (PIs) perform a similar function as comments in that they are not a textual part of an XML document but provide information to applications as to how the content should be processed. Processing instructions have the following form:

<?instruction options?>

15. What is XML documents?

- ✓ An XML document with correct syntax is called "Well Formed".
- ✓ A "Valid" XML document must also conform to a document type definition.

16. What are the building blocks of XML DTD?

The XML documents are made up by the following building blocks:

1. Elements
2. Attributes
3. Entities
4. PCDATA
5. CDATA

17. What is PCDATA?

- PCDATA means parsed character data.
- Think of character data as the text found between the start tag and the end tag of an XML element.
- PCDATA is text that WILL be parsed by a parser. The text will be examined by the parser for entities and markup.
- Tags inside the text will be treated as markup and entities will be expanded.

18. What is CDATA?

- CDATA means character data.
- CDATA is text that will NOT be parsed by a parser. Tags inside the text will NOT be treated as markup and entities will not be expanded.

19. Write the syntax of DTD?

The syntax of DTD is

<!DOCTYPE root_element SYSTEM | PUBLIC "filename" [internalDTDelements] >

✓ The exclamation mark **(!)** is used to signify the beginning of the declaration. ✓ **DOCTYPE** is the keyword used to denote this as a Document Type Definition.

- ✓ **root_element** is the name of the root element or document element of the XML document.
- ✓ **SYSTEM** and **PUBLIC** are keywords used to designate that the DTD is contained in an external document.
- ✓ The SYSTEM keyword is used in tandem with a URL to locate the DTD.
- ✓ The PUBLIC keyword specifies some public location that will usually be some application-specific resource reference.
- ✓ **internalDTDelements** are internal DTD declarations. These declarations will always be placed within opening ([) and closing (]) brackets.

20. Write the syntax of DTD element?

Each element in the DTD should be defined with the following syntax:

<!ELEMENT element_name rule >

- ✓ ELEMENT is the tag name that specifies that this is an element definition.
- ✓ element_name is the name of the element.
- ✓ rule is the definition to which the element's data content must conform.

21. Write the syntax of DTD attributes?

An attribute declaration has the following syntax:

<!ATTLIST element-name attribute-name attribute-type default-value>

22. What are the disadvantages of DTD?

- ✓ DTDs are composed of non-XML syntax
- ✓ It can only be a single DTD per document
- ✓ DTDS are not object oriented. There is no inheritance in DTDs
- ✓ DTDs do not support namespaces very well
- ✓ DTDs have weak data typing and no support for the XML DOM

23. Define XML Schema?

- An XML Schema describes the structure of an XML document.
- An XML document validated against an XML Schema is both "Well Formed" and "Valid". ▪ XML Schema is an XML-based (and more powerful) alternative to DTD.

24. List the data types supported in XML Schema?

XML Schemas is the support for data types.

- ✓ It is easier to describe allowable document content
- ✓ It is easier to validate the correctness of data
- ✓ It is easier to define data facets (restrictions on data)
- ✓ It is easier to define data patterns (data formats)
- ✓ It is easier to convert data between different data types

25. List the primitive data types in XML Schema?

- ✓ xs:string
- ✓ xs:decimal
- ✓ xs:integer
- ✓ xs:boolean
- ✓ xs:date
- ✓ xs:time

26. What is XML Namespaces?

- XML allows document authors to create custom elements.
- XML Namespaces provide a method to avoid element name conflicts.
- This extensibility can result in naming collisions (i.e. different elements that have the same name) among elements in an XML document.
- An XML namespace is a collection of element and attribute names. Each namespace has a unique name that provides a means for document authors to unambiguously refer to elements with the same name (i.e. prevent collisions).

27. How to define XML name spaces?

- When using prefixes in XML, a so-called namespace for the prefix must be defined.
- The namespace is defined by the xmlns attribute in the start tag of an element.
- The namespace declaration has the following syntax **xmlns:prefix="URI"**.

```
<root>
    <h:table xmlns:h="http://www.w3.org/TR/html4/">
        <h:tr>
            <h:td> Apples </h:td>
            <h:td> Bananas </h:td>
        </h:tr>
    </h:table>
</root>
```

28. Give the basic structure of XML schema? **(NOV 2016)**

The structure of XML schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="contact">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="name" type="xs:string" />
          <xs:element name="company" type="xs:string" />
          <xs:element name="phone" type="xs:int" />
        </xs:sequence>
      </xs:complexType>
    </xs:element>
</xs:schema>
```

29. What are the three types of XFiles?

The three types of XFiles are

i. XPath
ii. XLink
iii. XPointer

30. Define XPath?

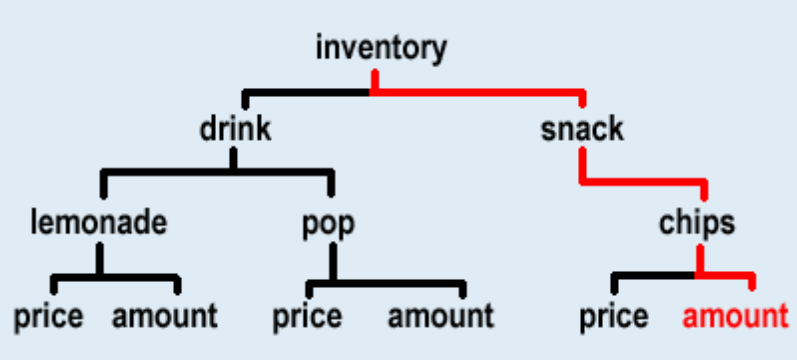
- The XML Path Language (XPath) is a standard for creating expressions that can be used to find specific pieces of information within an XML document.
- XPath is a syntax for defining parts of an XML document.
- XPath uses path expression to navigate in XML documents.
- XPath containms a library of standard functions.

31. What is XPath Expression?

- XPath expressions have the ability to locate nodes based on the nodes' type, name, or value or by the relationship of the nodes to other nodes within the XML document.
- An XPath expression describes the location of an element or attribute in our XML document. By starting at the root element, we can select any element in the document by carefully creating a chain of children elements. Each element is separated by a slash "/".
- For example, if we wanted to know the number of chips we have in stock(element amount) in lemonade2.xml, the XPath expression would be:XPath Expression:

  inventory/snack/chips/amount

XPath Trace:



32. What is the use of XPath Expression?

XPath expressions are used by both

- ✓ XSLT (for which XPath provides the core functionality) and
- ✓ XPointer to locate a set of nodes.

33. What type of value will be returned by XPath Expression?

The Xpath Expression can return any of the following:

- A node set
- A Boolean value
- A string value
- A numeric value

34. List the priority for evaluating XPath Expressions?

The priority for evaluating XPath expressions is as follows:

1. Grouping
2. Filters
3. Path operations

35. List the Operators and Special Characters used in XPath?

Operator and special characters	Description
/	Selects from the root node
//	Selects nodes in the document from the current node that match the selection
.	Selects the current node
..	Selects the parent of the current node
*	A wildcard character that selects all elements or attributes regardless of name
@	Selects an attribute
:	Namespace separator
()	Indicates a grouping within an XPath expression
[expression]	Indicates a filter expression
[n]	Indicates that the node with the specified index should be selected
+	Addition operator
-	Subtraction operator
Div	Division operator
*	Multiplication operator
Mod	Returns the remainder of a division operation

36. Write the syntax for XPath Expression?

- The XML Path Language provides a declarative notation, termed a pattern, used to select the desired set of nodes from XML documents. Each pattern describes a set of matching nodes to select from a hierarchical XML document. Each pattern describes a "navigation" path to the desired set of nodes similar to the Uniform Resource Identifier (URI) syntax.
- The basic syntax for an XPath expression

  axis::node test[predicate]

37. What is Location Path Expression?

- A location path can be absolute or relative path expression.
- An absolute location path starts with a slash (/) and a relative location path does not.
- In both cases the location path consists of one or more steps, each separated by a slash:

  An absolute location path

  /step/step/…

  An relative location path

  step/step/…

38. List the steps in location path expression?

1. An axis
2. A node test
3. A predicate

39. What is XPath Axes?

An axis defines the tree-relationship between the selected nodes and the current node.

The various Axis names are

- ✓ Ancestor
- ✓ ancestor-or-self
- ✓ attribute
- ✓ child
- ✓ descendant
- ✓ descendant-or-self
- ✓ following
- ✓ following-sibling
- ✓ namespace
- ✓ parent
- ✓ preceding
- ✓ preceding-sibling
- ✓ self

40. What is node test?

A node test identifies a node within an axis.

The various node test are

- ✓ comment()
- ✓ node()
- ✓ processing-instruction()
- ✓ text()

41. List the various XPath Functions?

XPath functions are used to evaluate XPath expressions and can be divided into one of four main groups:

1. Boolean
2. Node set
3. Number
4. String

42. What is meant by XPointer? **(APR 2017)**

- XPointer allows links to point to specific parts of an XML document.
- XPointer uses XPath expressions to navigate in the XML document.
- XPointer describes a location within an external document.
- XPointer can target a point within that XML document or a range within the target XML document.

43. List the XPointer Functions and their Return Location Sets?

Function	description
id()	Selects all nodes with the specified ID
root()	Selects the root element as the only location in a location set
here()	Selects the current element location in a location set
origin()	Selects the current element location for a node using an out-of-line link

44. What are the types of XPointer points?

The two different types of points can be represented using XPointer points:

1. Node points
2. Character points

45. List the XPointer Range Functions?

Function	Description
end-point()	Selects a location set consisting of the endpoints of the desired location steps
range-inside()	Selects the range(s) covering each location in the location-set argument
range-to()	Selects a range that completely covers the locations within the location-set argument
start-point()	Selects a location set consisting of the start points of the desired location steps

46. What is meant by XLink?

- XLink is used to create hyperlinks within XML documents.
- Any element in an XML document can behave as a link.
- With XLink, the links can be defined outside the linked files.

47. List the various XLink attributes?

Attribute	Description
xlink:type	This attribute must be specified and indicates what type of XLink is represented or defined.
xlink:href	This attribute contains the information necessary to locate the desired resource
xlink:role	This attribute describes the function of the link between the current resource and another.

xlink:arcrole	This attributes describes the function of the link between the current resource and another.
xlink:title	This attribute describes the meaning of the link between the resources.
xlink:show	This attribute indicates how the resource linked to should be displayed
xlink:actuate	This attribute specifies when to load the linked resource.
xlink:label	This attribute is used to identify a name for a target resource.
xlink:from	This attribute identifies the starting resource.
xlink:to	This attribute identifies the ending resource.

48. Mention any two difference between Xlink and XPath. **(NOV 2016)**

Xlink	XPath
➢ XLink is used to create hyperlinks within XML documents	➢ XPath is a syntax for defining parts of an XML documents
➢ Any element in an XML document can behaves as a link	➢ XPath uses path expressions to navigate in XML documents
➢ With XLink, the links can be defined outside the linked files.	➢ XPath contains a library of standard functions
	➢ XPath is a major element in the XSLT standard

49. What is XSL?

- XSL stands for eXtensible Stylesheet Language.
- The World Wide Web Consortium (W3C) started to develop XSL because there is need for an XML-based Stylesheet Language.

50. List the two languages of XSL technologies?

XSL has two independent languages:

- The XSL Transformation Language (XSLT)
- The XSL Formatting Object Language (XSL-FO)

XSLT is used to convert an XML document to another format. XSL-FO provides a way of describing the presentation of an XML document. Both technologies use a supporting XML technology.

51. What is XSLT?

- XSLT stands for XSL Transformations Language
- XSLT is a language used to transform an XML documents into another XML documents
- XSLT uses XPath to navigate in XML documents

52. Write the primary purpose of XSLT?

- XSLT is used to transform an XML document into another XML document, or another type of document that is recognized by a browser, like HTML and XHTML.
- XSLT can be used to add or remove elements and attributes to or from the output file.
- XSLT can also be used for rearranging and sorting elements. It can also be used for performing tests and making decisions about hiding and displaying of elements.

53. What is XSL-FO?

- XSL-FO stands for eXtensible Stylesheet Language-Formatting Objects.
- XSL-FO is a language for formatting XML data.
- It is an XML based markup languages describing the formatting of XML data for output to screen, paper or other media.

54. List the components XSL-FO documents?

An XSL-FO document can contain the following components:

- ✓ Page master
- ✓ Page master set
- ✓ Page sequences

55. What is DOM?

- The Document Object Model (DOM) provides a way of representing an XML document in memory so that it can be manipulated by your software.
- DOM is a standard application programming interface (API) that makes it easy for programmers to access elements and delete, add, or edit content and attributes.
- DOM was proposed by the World Wide Web Consortium (W3C) in August of 1997 in the User Interface Domain.

56. List the fundamentals interfaces of

DOM? The various DOM interfaces are

- ✓ Node
- ✓ NodeList
- ✓ NamedNodeMap
- ✓ Document
- ✓ DocumentFragment
- ✓ Element
- ✓ Attr
- ✓ CharacterData
- ✓ Text
- ✓ Comment

57. List the transversal interfaces of DOM?

- ✓ NodeIterator
- ✓ TreeWalker
- ✓ NodeFilter
- ✓ DocumentTraversal

58. What is SAX?

- SAX (Simple API for XML) is an event based parsing existing XML documents.
- SAX parser creates no parse tree.
- A parser that uses SAX parses the XML document sequentially. XML parsing is unidirectional.
- Memory used by a SAX parses is relatively low.
- The nature of SAX, the parsing is faster of an XML document.

59. Write the difference between SAX and DOM?

SAX	DOM
Parses node by node	Stores the entire XML document into memory before processing
Doesn't store the XML in memory	Occupies more memory
We can't insert or delete a node	We can insert or delete nodes
Top to bottom traversing	Traverse in any direction
SAX is an event based parser	DOM is a tree model parser
SAX is a Simple API for XML	Document Object Model (DOM) API
doesn't preserve comments	preserves comments
SAX generally runs a little faster than DOM	SAX generally runs a little faster than DOM

60. Give the benefits of SAX?

SAX is an easy-to-use API for parsing XML data. It's available in source and binary form free of charge. SAX has become one of the most popular tools for parsing XML due to its ease of use and widespread availability. Unlike DOM, SAX is an event-based parser. SAX reads XML serially and generates events when elements, text, comments and other data are found. SAX solves a large class of XML parsing problems easily and efficiently.

61. What are the types of XML database solutions?

The number of XML database solutions are

- ✓ database mapping
- ✓ native XML support

62. What is meant by JAXB?

- JAXB stands for Java Architecture for XML Binding
- JAXB provides a framework for representing XML documents as Java objects.
- JAXB is easier to use and a more efficient technique for processing XML documents than the SAX or DOM API.

63. Write the steps to create JAXB schema?

1. Review the database schema.
2. Construct the desired XML document.
3. Define a schema for the XML document.
4. Create the JAXB binding schema.
5. Generate the JAXB classes based on the schema.
6. Develop a Data Access Object (DAO).
7. Develop a servlet for HTTP access.

64. What is meant by XHTML?

- XHTML stands for E**X**tensible **H**yper **T**ext **M**arkup **L**anguage
- XHTML is almost identical to HTML
- XHTML is stricter than HTML
- XHTML is HTML defined as an XML application
- XHTML is supported by all major browsers

65. Write the syntax of XHTML?

- ✓ XHTML Must Be Well Formed
- ✓ All Elements and Attributes Must Be Lowercase
- ✓ Attribute Values Must Always Appear in Quotes
- ✓ Attributes May Not Be Minimized

66. What are the supported features of XHTML?

- ✓ Text Support
- ✓ Hyperlinks and Linking to Documents
- ✓ Table Support
- ✓ Forms Support
- ✓ Style Sheet Support
- ✓ Images Support

67. List the XHTML basic modules?

- ✓ Structure Module
- ✓ Text Module
- ✓ Hypertext Module

- ✓ List Module
- ✓ Basic Forms Module
- ✓ Basic Tables Module
- ✓ Image Module
- ✓ Object Module
- ✓ Meta-information Module
- ✓ Link Module
- ✓ Base Module

68. What is meant by XForms?

- XForms is the next generation of HTML forms
- XForms is richer and more flexible than HTML forms
- XForms is platform and device independent
- XForms separates data and logic from presentation
- XForms uses XML to define form data
- XForms stores and transports data in XML documents
- XForms contains features like calculations and validations of forms
- XForms reduces or eliminates the need for scripting

69. What are the three layers in XForms?

XForms forms technology is split into three layers. These layers are

1. Purpose layer
2. Presentation layer
3. Data layer

11 MARKS

1. Explain in detail about XML? **(5 MARKS) (NOV 2016) (APR 2017)**

Introduction to XML:

XML is a software- and hardware-independent tool for storing and transporting data.

What is XML?

- XML stands for EXtensible Markup Language
- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive
- XML is a W3C Recommendation

XML Does Not DO Anything

This note is a note to Tove, from Jani, stored as XML:

```
<note>
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
</note>
```

The note is quite self-descriptive. It has sender and receiver information. It also has a heading and a message body.

But still, this XML document does not DO anything. XML is just information wrapped in tags. Someone must write a piece of software to send, receive, store, or display it:

Note

To: Tove

From: Jani

Reminder

Don't forget me this weekend!

The Difference Between XML and HTML

XML and HTML were designed with different goals:

- XML was designed to carry data - with focus on what data is
- HTML was designed to display data - with focus on how data looks
- XML tags are not predefined like HTML tags are

XML Does Not Use Predefined Tags

- The XML language has no predefined tags.
- The tags in the example above (like <to> and <from>) are not defined in any XML standard. These tags are "invented" by the author of the XML document.
- HTML works with predefined tags like <p>, <h1>, <table>, etc.
- With XML, the author must define both the tags and the document structure.

XML is Extensible

- Most XML applications will work as expected even if new data is added (or removed).
- Imagine an application designed to display the original version of note.xml (<to> <from> <heading> <data>).
- Then imagine a newer version of note.xml with added <date> and <hour> elements, and a removed <heading>.

The way XML is constructed; older version of the application can still work:

```
<note>
    <date> 2015-09-01 </date>
    <hour> 08:30 </hour>
    <to> Tove </to>
    <from> Jani </from>
    <body> Don't forget me this weekend!
</body> </note>
```

Note
To: Tove
From: Jani
Date: 2015-09-01 08:30
Head: (none)
Don't forget me this weekend!

XML Simplifies Things

- It simplifies data sharing
- It simplifies data transport
- It simplifies platform changes
- It simplifies data availability

✓ Many computer systems contain data in incompatible formats. Exchanging data between incompatible systems (or upgraded systems) is a time-consuming task for web developers. Large amounts of data must be converted, and incompatible data is often lost.

- ✓ XML stores data in plain text format. This provides a software- and hardware-independent way of storing, transporting, and sharing data.
- ✓ XML also makes it easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.
- ✓ With XML, data can be available to all kinds of "reading machines" like people, computers, voice machines, news feeds, etc.

XML is a W3C Recommendation

XML became a W3C Recommendation on February 10, 1998.

2. Explain the basic concept of XML? (5 MARKS) (APR 2017)

XML Separates Data from Presentation

- XML does not carry any information about how to be displayed.
- The same XML data can be used in many different presentation scenarios.
- Because of this, with XML, there is a full separation between data and presentation.

XML is Often a Complement to HTML

- In many HTML applications, XML is used to store or transport data, while HTML is used to format and display the same data.

XML Separates Data from HTML

- When displaying data in HTML, you should not have to edit the HTML file when the data changes.
- With XML, the data can be stored in separate XML files.

With a few lines of JavaScript code, you can read an XML file and update the data content of any HTML page.

Display Books.xml

Title	Author
Everyday Italian	Giada De Laurentiis
Harry Potter	J K. Rowling
XQuery Kick Start	James McGovern
Learning XML	Erik T. Ray

Books.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book category="cooking">
        <title lang="en">Everyday Italian </title>
        <author> Giada De Laurentiis </author>
        <year> 2005 </year>
        <price> 30.00 </price>
    </book>
    <book category="children">
        <title lang="en"> Harry Potter </title>
        <author> J K. Rowling </author>
        <year> 2005 </year>
        <price> 29.99 </price>
    </book>
    <book category="web">
        <title lang="en"> XQuery Kick Start </title>
        <author> James McGovern </author>
        <author> Per Bothner </author>
        <author> Kurt Cagle </author>
        <author> James Linn </author>
        <author> Vaidyanathan Nagarajan </author>
        <year> 2003 </year>
        <price> 49.99 </price>
    </book>
    <book category="web" cover="paperback">
        <title lang="en"> Learning XML </title>
        <author> Erik T. Ray </author>
        <year> 2003 </year>
        <price> 39.95 </price>
    </book>
</bookstore>
```

Transaction Data

Thousands of XML formats exist, in many different industries, to describe day-to-day data transactions:

- Stocks and Shares
- Financial transactions
- Medical data
- Mathematical data
- Scientific measurements
- News information

Example: XML News

XML News is a specification for exchanging news and other information.

Using a standard makes it easier for both news producers and news consumers to produce, receive, and archive any kind of news information across different hardware, software, and programming languages.

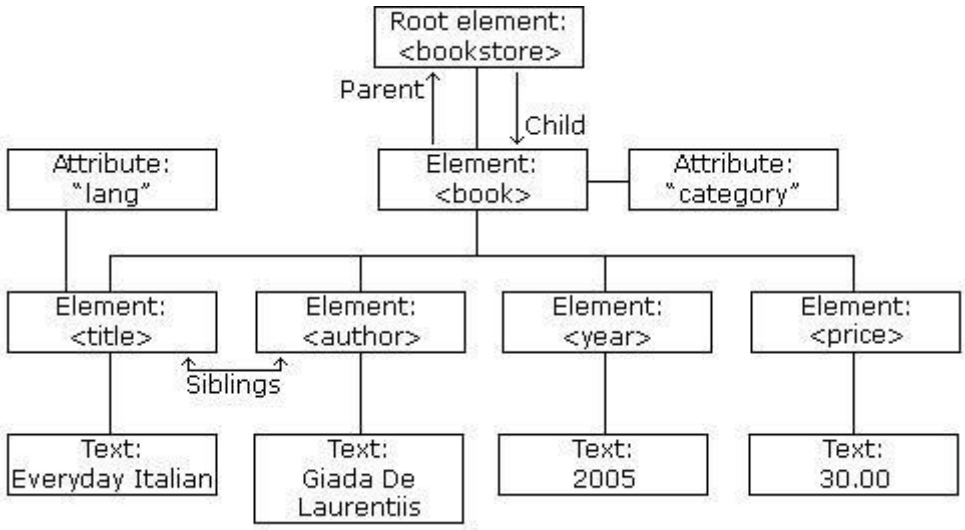
An example XML News document:

```
<?xml version="1.0" encoding="UTF-8"?>
<nitf>
    <head>
        <title> Colombia Earthquake </title>
    </head>
 <body>
    <headline>
        <hl1> 143 Dead in Colombia Earthquake </hl1>
    </headline>
    <byline>
        <bytag> By Jared Kotler, Associated Press Writer </bytag>
    </byline>
    <dateline>
        <location> Bogota, Colombia </location>
        <date> Monday January 25 1999 7:28 ET </date>
    </dateline>
 </body>
</nitf>
```

XML Tree:

XML documents form a tree structure that starts at "the root" and branches to "the leaves".

XML Tree Structure



An Example XML Document

The image above represents books in this XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book category="cooking">
        <title lang="en"> Everyday Italian </title>
        <author> Giada De Laurentiis </author>
        <year> 2005 </year>
        <price> 30.00 </price>
    </book>
        <book category="children">
        <title lang="en"> Harry Potter </title>
        <author> J K. Rowling </author>
        <year> 2005 </year>
        <price> 29.99 </price>
    </book>
    <book category="web">
        <title lang="en"> Learning XML </title>
        <author> Erik T. Ray </author>
        <year> 2003 </year>
        <price> 39.95 </price>
    </book>
</bookstore>
```

XML Tree Structure

- XML documents are formed as **element trees**.
- An XML tree starts at a **root element** and branches from the root to **child elements**.

All elements can have sub elements (child elements):

```
<root>
  <child>
        <subchild>.....</subchild>
  </child>
</root>
```

Self-Describing Syntax

- XML uses a much self-describing syntax.

A prolog defines the XML version and the character encoding:

```
<?xml version="1.0" encoding="UTF-8"?>
```

The next line is the **root element** of the document:

```
<bookstore>
```

The next line starts a <book> element:

```
<book category="cooking">
```

The <book> elements have **4 child elements**: <title>,< author>, <year>, <price>.

```
<title lang="en"> Everyday Italian </title>
<author> Giada De Laurentiis </author>
<year> 2005 </year>
<price> 30.00 </price>
```

The next line ends the book element:

```
</book>
```

3. Explain in detail about XML syntax? **(6 MARKS) (NOV 2016)**

- The syntax rules of XML are very simple and very strict.
- The rules are very easy to learn, and very easy to use.

XML Documents Must Have a Root Element

XML documents must contain one root element that is the parent of all other elements:

```
<root>
  <child>
        <subchild>.... </subchild>
  </child>
</root>
```

In this example **<note>** is the root element:

```
<? xml version="1.0" encoding="ISO-8859-1"?>
    <note>
        <to> Tove </to>
        <from> Jani </from>
        <heading> Reminder </heading>
        <body> Don't forget me this weekend! </body>
</note>
```

The XML Prolog

This line is called the XML **prolog**:

```
<?xml version="1.0" encoding="UTF-8"?>
```

- The XML prolog is optional. If it exists, it must come first in the document.
- XML documents can contain international characters, like Norwegian øæå or French êèé.
- To avoid errors, you should specify the encoding used, or save your XML files as UTF-8.
- UTF-8 is the default character encoding for XML documents.

All XML Elements Must Have a Closing Tag

- In HTML, some elements might work well, even with a missing closing tag:

```
<p>This is a paragraph.
<br>
```

- In XML, it is illegal to omit the closing tag. All elements **must** have a closing tag:

```
<p> This is a paragraph. </p>
<br />
```

The XML declaration does not have a closing tag. This is not an error. The declaration is not a part of XML.

XML Tags are Case Sensitive

- XML tags are case sensitive. The tag <Letter> is different from the tag <letter>.
- Opening and closing tags must be written with the same case:

<Message> This is incorrect </message>

<message> This is correct </message>

"Opening and closing tags" are often referred to as "Start and end tags". Use whatever you prefer. It is exactly the same thing.

XML Elements Must be Properly Nested

- In HTML, you might see improperly nested elements:

  This text is bold and italic

- In XML, all elements **must** be properly nested within each other:

  This text is bold and italic

In the example above, "Properly nested" simply means that since the element is opened inside the element, it must be closed inside the element.

XML Attribute Values Must be Quoted

- XML elements can have attributes in name/value pairs just like in HTML.
- In XML, the attribute values must always be quoted.

Incorrect:

```
<note date=12/11/2007>
        <to> Tove </to>
        <from> Jani </from>
</note>
```

Correct:

```
<note date="12/11/2007">
        <to> Tove </to>
        <from> Jani </from>
</note>
```

The error in the first document is that the date attribute in the note element is not quoted.

Entity References

- Some characters have a special meaning in XML.
- If you place a character like "<" inside an XML element, it will generate an error because the parser interprets it as the start of a new element.

This will generate an XML error:

<message> salary < 1000 </message>

To avoid this error, replace the "<" character with an **entity reference**:

<message> salary < 1000 </message>

There are 5 pre-defined entity references in XML:

<	<	less than
>	>	greater than
&	&	ampersand
'	'	apostrophe
"	"	quotation mark

Comments in XML

The syntax for writing comments in XML is similar to that of HTML.

<!-- This is a comment -->

Two dashes in the middle of a comment are not allowed.

Not allowed:

<!-- This is a -- comment -->

Strange, but allowed:

<!-- This is a - - comment -->

White-space is Preserved in XML

- XML does not truncate multiple white-spaces (HTML truncates multiple white-spaces to one single white-space):

  XML: Hello Tove

  HTML: Hello Tove

XML Stores New Line as LF

- Windows applications store a new line as: carriage return and line feed (CR+LF).
  - ✓ Unix and Mac OSX uses LF.
  - ✓ Old Mac systems uses CR.
  - ✓ XML stores a new line as LF.

Well Formed XML

- XML documents that conform to the syntax rules above are said to be "Well Formed" XML documents.

4. Discuss about working with xml elements and attributes? (11 MARKS) (NOV 2016)

XML ELEMENTS:

- An XML document contains XML Elements.
- An XML element is everything from (including) the element's start tag to (including) the element's end tag.

  <price> 29.99 **</price>**

An elements may contain:

- ✓ text
- ✓ attributes
- ✓ other elements
- ✓ or a mix of the above

Example

```
<bookstore>
    <book category = "Novel" >
        <title> Harry Potter </title>
        <author> J K. Rowling </author>
        <year> 2005 </year>
        <price> 29.99 </price>
    </book>
    <book category = "WEB" >
        <title> Learning XML </title>
        <author> Erik T. Ray </author>
        <year> 2003 </year>
        <price> 39.95 </price>
    </book>
</bookstore>
```

In the example above,

- <bookstore> - Root Element
- <book> have element contents, because they contain other elements.
- <book> also has an attribute (category="CHILDREN").
- <title>, <author>, <year>, and <price> have text content because they contain text.

Empty XML Elements

- An element with no content is said to be empty.
- In XML, you can indicate an empty element like this: <element> </element>
- You can also use a so called self-closing tag: <element />

XML Naming Rules

XML elements must follow these naming rules:

- ✓ Element names are case-sensitive
- ✓ Element names must start with a letter or underscore
- ✓ Element names cannot start with the letters xml (or XML, or Xml, etc)
- ✓ Element names can contain letters, digits, hyphens, underscores, and periods
- ✓ Element names cannot contain spaces

XML Elements are Extensible

XML elements can be extended to carry more information.

Example

```
<?xml version="1.0" encoding="ISO-8859-1"?>
    <note>
        <to> Tove </to>
        <from> Jani </from>
        <body> Don't forget me this weekend! </body>
    </note>
```

The XML document to produce this output:

MESSAGE

To: Tove

From: Jani

Don't forget me this weekend!

The XML document added some extra information to it:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
    <note>
        <date> 2008-01-10 </date>
        <to> Tove </to>
        <from> Jani </from>
        <heading> Reminder </heading>
        <body> Don't forget me this weekend! </body>
    </note>
```

XML ATTRIBUTES:

- XML elements can have attributes in the start tag, just like HTML.
- Attributes are used to provide additional information about elements.

XML Attributes Must be Quoted

- Attribute values must always be quoted.
- Either single or double quotes can be used.

For a person's gender, the person element can be written like this:

```
<person gender = "male" >
```

or like this:

```
<person gender = 'male' >
```

If the attribute value itself contains double quotes you can use single quotes, like in this example:

```
<gangster name = 'George "Shotgun" Ziegler' >
```

or you can use character entities:

```
<gangster name = "George &quot; Shotgun &quot; Ziegler" >
```

XML Elements vs. Attributes

Take a look at these examples:

```
<person gender="female">
    <firstname> Anna </firstname>
    <lastname> Smith </lastname>
</person>

<person>
    <gender> female </gender>
    <firstname> Anna </firstname>
    <lastname> Smith </lastname>
</person>
```

In the first example gender is an attribute. In the last, gender is an element. Both examples provide the same information.

There are no rules about when to use attributes or when to use elements in XML.

The following three XML documents contain exactly the same information:

A date attribute is used in the **first example**:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<note date="10/01/2008" >
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
</note>
```

A date element is used in the **second example**:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<note>
    <date> 10/01/2008 </date>
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
</note>
```

An expanded date element is used in the **third**:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<note>
    <date>
        <day> 10 </day>
        <month> 01 </month>
        <year> 2008 </year>
    </date>
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
</note>
```

Should you avoid using attributes?

Consider when using attributes are:

- ✓ attributes cannot contain multiple values (elements can)
- ✓ attributes cannot contain tree structures (elements can)
- ✓ attributes are not easily expandable (for future changes)
- ✓ Attributes are more difficult to manipulate by program code.
- ✓ Attributes are difficult to read and maintain
- ✓ Use elements for data.

Don't end up like this:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<note day="10" month="01" year="2008"
    to="Tove" from="Jani" heading="Reminder"
    body="Don't forget me this weekend!">
</note>
```

XML Attributes for Metadata

- Sometimes ID references are assigned to elements.
- These IDs can be used to identify XML elements in much the same way as the id attribute in HTML.

Example

```
<messages>
        <note id="501">
                <to> Tove </to>
                <from> Jani </from>
                <heading> Reminder </heading>
                <body> Don't forget me this weekend! </body>
        </note>
        <note id="502">
                <to> Jani </to>
                <from> Tove </from>
                <heading> Re: Reminder </heading>
                <body> I will not </body>
                </note>
</messages>
```

5. Explain in detail about XML Documents? **(6 MARKS)**

XML Document Types:

1. An XML document with correct syntax is called "Well Formed".
2. A "Valid" XML document must also conform to a document type definition.

(i) Well Formed XML Documents:

- XML with correct syntax is Well Formed XML.

The syntax rules

- ✓ XML documents must have a root element
- ✓ XML elements must have a closing tag
- ✓ XML tags are case sensitive
- ✓ XML elements must be properly nested
- ✓ XML attribute values must be quoted.

Example

```
<?xml version="1.0" encoding="ISO-8859-1"?>
    <note>
        <to> Tove </to>
        <from> Jani </from>
        <heading> Reminder </heading>
        <body> Don't forget me this weekend! </body>
    </note>
```

(ii) Valid XML Documents

- A "valid" XML document is not the same as a "well formed" XML document.
- A "valid" XML document must be well formed. In addition it must conform to a document type definition.
- Rules that define the legal elements and attributes for XML documents are called Document Type Definitions (DTD) or XML Schemas.

There are two different document type definitions that can be used with XML:

1. DTD - The original Document Type Definition
2. XML Schema - An XML-based alternative to DTD

(ii) XML DTD

- An XML document with correct syntax is called "Well Formed".
- An XML document validated against a DTD is both "Well Formed" and "Valid".

Valid XML Documents

A "Valid" XML document is a "Well Formed" XML document, which also conforms to the rules of a DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
</note>
```

The DOCTYPE declaration, in the example above, is a reference to an external DTD file. The content of the file is shown in the paragraph below.

- The purpose of a DTD is to define the structure of an XML document. It defines the structure with a list of legal elements:

```
<!DOCTYPE note[
    <!ELEMENT note (to,from,heading,body)>
    <!ELEMENT to (#PCDATA)> <!ELEMENT
    from (#PCDATA)> <!ELEMENT heading
    (#PCDATA)> <!ELEMENT body
    (#PCDATA)>
]>
```

The DTD above is interpreted like this:

- !DOCTYPE note defines that the root element of the document is note
- !ELEMENT note defines that the note element must contain the elements: "to, from, heading, body"
- !ELEMENT to defines the to element to be of type "#PCDATA"
- !ELEMENT from defines the from element to be of type "#PCDATA"
- !ELEMENT heading defines the heading element to be of type "#PCDATA"
- !ELEMENT body defines the body element to be of type "#PCDATA"
- #PCDATA means parse-able text data.

Using DTD for Entity Declaration

- A doctype declaration can also be used to define special characters and character strings, used in the document:

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note [
    <!ENTITY nbsp " ">
    <!ENTITY writer "Writer: Donald Duck.">
    <!ENTITY copyright "Copyright: W3Schools.">
]>
<note>
    <to> Tove </to>
    <from> Jani </from>
    <heading> Reminder </heading>
    <body> Don't forget me this weekend! </body>
    <footer> &writer; &copyright; </footer>
</note>
```

The DTD above is interpreted like this:

- ✓ **!DOCTYPE note** defines that the root element of this document is note
- ✓ **!ELEMENT note** defines that the note element must contain four elements: "to,from,heading,body"
- ✓ **!ELEMENT to** defines the to element to be of type "#PCDATA"
- ✓ **!ELEMENT from** defines the from element to be of type "#PCDATA"
- ✓ **!ELEMENT heading** defines the heading element to be of type "#PCDATA"
- ✓ **!ELEMENT body** defines the body element to be of type "#PCDATA"

6. Explain in detail about Document Type Definition (DTD)? (5 MARKS)

- A DTD is a Document Type Definition.
- A DTD defines the structure and the legal elements and attributes of an XML document.

Why Use a DTD?

- With a DTD, independent groups of people can agree on a standard DTD for interchanging data.
- An application can use a DTD to verify that XML data is valid.

(i) An Internal DTD Declaration

If the DTD is declared inside the XML file, it must be wrapped inside the <!DOCTYPE> definition:

XML document with an internal DTD

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
   <to> Tove </to>
   <from> Jani </from>
   <heading> Reminder </heading>
   <body> Don't forget me this weekend </body>
</note>
```

(ii) An External DTD Declaration

If the DTD is declared in an external file, the <!DOCTYPE> definition must contain a reference to the DTD file:

XML document with a reference to an external DTD

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
        <note>
                <to> Tove </to>
                <from> Jani </from>
                <heading> Reminder </heading>
                <body> Don't forget me this weekend! </body>
        </note>
```

And here is the file "note.dtd", which contains the

```
DTD: <!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

7. Explain the building blocks of XML DTD? **(6 MARKS)**

The main building blocks of both XML and HTML documents are elements.

The Building Blocks of XML Documents:

The XML documents are made up by the following building blocks:

1. Elements
2. Attributes
3. Entities
4. PCDATA
5. CDATA

1. Elements

- Elements are the **main building blocks** of both XML and HTML documents.
- Examples of HTML elements are "body" and "table".
- Examples of XML elements could be "note" and "message".
- Elements can contain text, other elements, or be empty.
- Examples of empty HTML elements are "hr", "br" and "img".

Examples

<body> some text </body>

<message> some text </message>

2. Attributes

- Attributes provide **extra information about elements**.
- Attributes are always placed inside the opening tag of an element.
- Attributes always come in name/value pairs.

The following "img" element has additional information about a source file:

****

The name of the element is "img". The name of the attribute is "src". The value of the attribute is "computer.gif". Since the element itself is empty it is closed by a " /".

3. Entities

- Some characters have a special meaning in XML, like the less than sign (<) that defines the start of an XML tag.
- Most of you know the HTML entity: " ". This "no-breaking-space" entity is used in HTML to insert an extra space in a document. Entities are expanded when a document is parsed by an XML parser.

The following entities are predefined in XML:

Entity References	Character
<	<
>	>
&	&
"	"
'	'

4. PCDATA

- **PCDATA** means parsed character data.
- Think of character data as the text found between the start tag and the end tag of an XML element.
- PCDATA is text that WILL be parsed by a parser. The text will be examined by the parser for entities and markup.
- Tags inside the text will be treated as markup and entities will be expanded.
- However, parsed character data should not contain any &, <, or > characters; these need to be represented by the & < and > entities, respectively.

5. CDATA

- CDATA means character data.
- **CDATA is text that will NOT be parsed by a parser**. Tags inside the text will NOT be treated as markup and entities will not be expanded.

8. Explain the XML DTD Elements in detail? (6 MARKS)

DTD – Elements:

In a DTD, elements are declared with an ELEMENT declaration.

1. Declaring Elements

In a DTD, XML elements are declared with an element declaration with the following syntax:

<! ELEMENT element-name category>

or

<! ELEMENT element-name (element-content)>

2. Empty Elements

Empty elements are declared with the category keyword EMPTY:

<! ELEMENT element-name EMPTY>

Example:

<! ELEMENT br EMPTY>

XML example:

3. Elements with Parsed Character Data

Elements with only parsed character data are declared with #PCDATA inside parentheses:

<! ELEMENT element-name (#PCDATA)>

Example:

<! ELEMENT from (#PCDATA)>

4. Elements with any Contents

Elements declared with the category keyword ANY, can contain any combination of parsable data:

<! ELEMENT element-name ANY>

Example:

<! ELEMENT note ANY>

5. Elements with Children (sequences)

Elements with one or more children are declared with the name of the children elements inside parentheses:

<! ELEMENT element-name (child1)>

or

<! ELEMENT element-name (child1, child2...)>

Example:

<! ELEMENT note (to,from,heading,body)>

- When children are declared in a sequence separated by commas, the children must appear in the same sequence in the document.
- In a full declaration, the children must also be declared, and the children can also have children.
- The full declaration of the "note" element is:

<! ELEMENT note (to, from, heading,
body)> <! ELEMENT to (#PCDATA)>
<! ELEMENT from (#PCDATA)> <!
ELEMENT heading (#PCDATA)>
<! ELEMENT body (#PCDATA)>

6. Declaring Only One Occurrence of an Element

<! ELEMENT element-name (child-name)>

Example:

<! ELEMENT note (message)>

The example above declares that the child element "message" must occur once, and only once inside the "note" element.

7. Declaring Minimum One Occurrence of an Element

<! ELEMENT element-name (child-name+)>

Example:

<! ELEMENT note (message+)>

The + sign in the example above declares that the child element "message" must occur one or more times inside the "note" element.

8. Declaring Zero or More Occurrences of an Element

<! ELEMENT element-name (child-name*)>

Example:

<! ELEMENT note (message*)>

The * sign in the example above declares that the child element "message" can occur zero or more times inside the "note" element.

9. Declaring Zero or One Occurrences of an Element

<! ELEMENT element-name (child-name?)>

Example:

<! ELEMENT note (message?)>

- The? Sign in the example above declares that the child element "message" can occur zero or one time inside the "note" element.

10. Declaring either/or Content

<! ELEMENT note (to,from,header,(message|body))>

- The example above declares that the "note" element must contain a "to" element, a "from" element, a "header" element, and either a "message" or a "body" element.

11. Declaring Mixed Content

<! ELEMENT note (#PCDATA|to|from|header|message)*>

- The example above declares that the "note" element can contain zero or more occurrences of parsed character data, "to", "from", "header", or "message" elements.

9. Explain in detail XML Schema? **(11 MARKS)**

- An XML Schema describes the structure of an XML document.
- The XML Schema language is also referred to as XML Schema Definition (XSD).

XSD Example

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="note">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="to" type="xs:string"/>
                <xs:element name="from" type="xs:string"/>
                <xs:element name="heading" type="xs:string"/>
                <xs:element name="body" type="xs:string"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

The purpose of an XML Schema is to define the legal building blocks of an XML document:

- the elements and attributes that can appear in a document
- the number of (and order of) child elements
- data types for elements and attributes
- default and fixed values for elements and attributes

Why Learn XML Schema?

- In the XML world, hundreds of standardized XML formats are in daily use.
- Many of these XML standards are defined by XML Schemas.
- XML Schema is an XML-based (and more powerful) alternative to DTD.

XML Schemas Support Data Types

XML Schemas is the support for data types.

- It is easier to describe allowable document content
- It is easier to validate the correctness of data
- It is easier to define data facets (restrictions on data)
- It is easier to define data patterns (data formats)
- It is easier to convert data between different data types

XML Schemas use XML Syntax

XML Schemas is that they are written in XML.

- You don't have to learn a new language
- You can use your XML editor to edit your Schema files
- You can use your XML parser to parse your Schema files
- You can manipulate your Schema with the XML DOM
- You can transform your Schema with XSLT

XML Schemas are extensible, because they are written in XML.

With an extensible Schema definition you can:

- Reuse your Schema in other Schemas
- Create your own data types derived from the standard types
- Reference multiple schemas in the same document

XML Schemas Secure Data Communication

- When sending data from a sender to a receiver, it is essential that both parts have the same "expectations" about the content.
- With XML Schemas, the sender can describe the data in a way that the receiver will understand.
- A date like: "03-11-2004" will, in some countries, be interpreted as 3.November and in other countries as 11.March.

However, an XML element with a data type like this:

```
<date type="date">2004-03-11</date>
```

ensures a mutual understanding of the content, because the XML data type "date" requires the format "YYYY-MM-DD".

Well-Formed is Not Enough

A well-formed XML document is a document that conforms to the XML syntax rules, like:

- it must begin with the XML declaration
- it must have one unique root element
- start-tags must have matching end-tags
- elements are case sensitive
- all elements must be closed
- all elements must be properly nested
- all attribute values must be quoted
- entities must be used for special characters

XSD How To?

- XML documents can have a reference to a DTD or to an XML Schema.

A simple XML document called "note.xml":

```
<?xml version="1.0"?>
<note>
        <to> Tove </to>
        <from> Jani </from>
        <heading> Reminder </heading>
        <body> Don't forget me this weekend! </body>
</note>
```

An XML Schema:

The following example is an XML Schema file called "note.xsd" that defines the elements of the XML document above ("note.xml"):

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.w3schools.com"
xmlns="http://www.w3schools.com"
elementFormDefault="qualified"> <xs:element name="note">

 <xs:complexType>
  <xs:sequence>
   <xs:element name="to" type="xs:string"/>
   <xs:element name="from" type="xs:string"/>
   <xs:element name="heading" type="xs:string"/>
   <xs:element name="body" type="xs:string"/>
  </xs:sequence>
 </xs:complexType>
</xs:element>
</xs:schema>
```

The note element is a complex type because it contains other elements. The other elements (to, from, heading, body) are simple types because they do not contain other elements.

A Reference to an XML Schema

This XML document has a reference to an XML Schema:

```
<?xml version="1.0"?>
<note
xmlns="http://www.w3schools.com"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://www.w3schools.com
note.xsd"> <to> Tove </to>
 <from> Jani </from>
 <heading> Reminder </heading>
 <body> Don't forget me this weekend! </body>
</note>
```

XSD - The <schema> Element:

The <schema> element is the root element of every XML Schema.

```
<?xml version="1.0"?>
<xs:schema>
    ...
    ...
</xs:schema>
```

The <schema> element may contain some attributes. A schema declaration often looks something like this:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.w3schools.com"
xmlns="http://www.w3schools.com"
elementFormDefault="qualified"> ...

...
</xs:schema>
```

The following fragment:

xmlns:xs=http://www.w3.org/2001/XMLSchema

indicates that the elements and data types used in the schema come from the "http://www.w3.org/2001/XMLSchema" namespace. It also specifies that the elements and data types that come from the "http://www.w3.org/2001/XMLSchema" namespace should be prefixed with **xs:**

This fragment:

targetNamespace=http://www.w3schools.com

indicates that the elements defined by this schema (note, to, from, heading, body.) come from the "http://www.w3schools.com" namespace.

This fragment:

elementFormDefault="qualified"

indicates that any elements used by the XML instance document which were declared in this schema must be namespace qualified.

Referencing a Schema in an XML Document

This XML document has a reference to an XML Schema:

```
<?xml version="1.0"?>
<note xmlns="http://www.w3schools.com"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.w3schools.com note.xsd">

<to> Tove </to>
<from> Jani </from>
<heading> Reminder </heading>
<body> Don't forget me this weekend! </body>
</note>
```

10. Explain in detail about XML Schema Elements and Attributes? (6 MARKS)

XML Schema Elements:

- XML Schemas define the elements of your XML files.
- A simple element is an XML element that contains only text. It cannot contain any other elements or attributes.

Defining a Simple Element

The syntax for defining a simple element is:

<xs:element name="xxx" type="yyy"/>

where xxx is the name of the element and yyy is the data type of the element.

XML Schema has a lot of built-in data types. The most common types are:

- ✓ xs:string
- ✓ xs:decimal
- ✓ xs:integer
- ✓ xs:boolean
- ✓ xs:date
- ✓ xs:time

Example

```
<lastname> Refsnes </lastname>
<age> 36 </age>
<dateborn> 1970-03-27 </dateborn>
```

The simple element definitions:

```
<xs:element name="lastname" type="xs:string"/>
<xs:element name="age" type="xs:integer"/>
<xs:element name="dateborn" type="xs:date"/>
```

Default and Fixed Values for Simple Elements

Simple elements may have a default value OR a fixed value specified.

- A **default value** is automatically assigned to the element when no other value is specified. In the following example the default value is "red":

```
<xs:element name="color" type="xs:string" default="red"/>
```

- A **fixed value** is also automatically assigned to the element, and you cannot specify another value.

In the following example the fixed value is "red":

```
<xs:element name="color" type="xs:string" fixed="red"/>
```

XSD Attributes:

Simple elements cannot have attributes. If an element has attributes, it is considered to be of a complex type. But the attribute itself is always declared as a simple type.

The syntax for defining an attribute is:

```
<xs:attribute name="xxx" type="yyy"/>
```

where xxx is the name of the attribute and yyy specifies the data type of the attribute.

Example

An XML element with an attribute:

```
<lastname lang="EN">Smith</lastname>
```

the corresponding attribute definition

```
<xs:attribute name="lang" type="xs:string"/>
```

Default and Fixed Values for Attributes

Attributes may have a default value OR a fixed value specified.

- A **default value** is automatically assigned to the attribute when no other value is specified. In the following example the default value is "EN":

```
<xs:attribute name="lang" type="xs:string" default="EN"/>
```

- A **fixed value** is also automatically assigned to the attribute, and you cannot specify another value.

In the following example the fixed value is "EN":

```
<xs:attribute name="lang" type="xs:string" fixed="EN"/>
```

Optional and Required Attributes

Attributes are optional by default. To specify that the attribute is required, use the "use" attribute:

```
<xs:attribute name="lang" type="xs:string" use="required"/>
```

11. Explain in detail about XML Namespace? (11 MARKS) (NOV 2017)

XML Namespaces provide a method to avoid element name conflicts.

Name Conflicts

In XML, element names are defined by the developer. This often results in a conflict when trying to mix XML documents from different XML applications.

This XML carries HTML table information:

```
<table>
    <tr>
        <td> Apples </td>
        <td> Bananas </td>
    </tr>
</table>
```

This XML carries information about a table (a piece of furniture):

```
<table>
    <name> African Coffee Table </name>
    <width> 80 </width>
    <length> 120 </length>
</table>
```

Solving the Name Conflict Using a Prefix

Name conflicts in XML can easily be avoided using a name prefix.

This XML carries information about an HTML table, and a piece of furniture:

```
<h:table>
  <h:tr>
    <h:td> Apples </h:td>
    <h:td> Bananas </h:td>
  </h:tr>
</h:table>
<f:table>
    <f:name> African Coffee Table </f:name>
    <f:width> 80 </f:width>
    <f:length> 120 </f:length>
</f:table>
```

In the example above, there will be no conflict because the two <table> elements have different names.

XML Namespaces - The xmlns Attribute

- When using prefixes in XML, a **namespace** for the prefix must be defined.
- The namespace can be defined by an **xmlns** attribute in the start tag of an element.
- The namespace declaration has the following syntax **xmlns:prefix="URI".**

```
<root>
    <h:table xmlns:h="http://www.w3.org/TR/html4/">
      <h:tr>
            <h:td> Apples </h:td>
            <h:td> Bananas </h:td>
      </h:tr>
    </h:table>
    <f:table xmlns:f="http://www.w3schools.com/furniture">
            <f:name> African Coffee Table </f:name>
            <f:width> 80 </f:width>
             <f:length> 120 </f:length>
    </f:table>
</root>
```

In the example above:

- The xmlns attribute in the first <table> element gives the h: prefix a qualified namespace.
- The xmlns attribute in the second <table> element gives the f: prefix a qualified namespace.
- When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace.

Namespaces can also be declared the XML root element:

```
<root xmlns:h="http://www.w3.org/TR/html4/"
   xmlns:f="http://www.w3schools.com/furniture">
<h:table>
        <h:tr>
            <h:td> Apples </h:td>
             <h:td> Bananas </h:td>
        </h:tr>
</h:table>
<f:table>
        <f:name> African Coffee Table </f:name>
        <f:width> 80 </f:width>
        <f:length> 120 </f:length>
</f:table>
</root>
```

- The namespace URI is not used by the parser to look up information.
- The purpose of using an URI is to give the namespace a unique name.
- However, companies often use the namespace as a pointer to a web page containing namespace information.

Default Namespaces:

Defining a default namespace for an element saves us from using prefixes in all the child elements.

It has the following syntax:

xmlns="*namespaceURI*"

This XML carries HTML table information:

```
<table xmlns="http://www.w3.org/TR/html4/">
        <tr>
            <td>Apples</td>
            <td>Bananas</td>
        </tr>
</table>
```

12. Explain in detail about XLink and its attributes? **(11 MARKS)**

- XLink is used to create hyperlinks within XML documents.
- Any element in an XML document can behave as a link.
- With XLink, the links can be defined outside the linked files.

The anchor element, <a>, within HTML indicates a link to another resource on an HTML page. This could be a location within the same document or a document located elsewhere. In HTML terms, the anchor element creates a hyperlink to another location.

The hyperlink can either appear as straight text, a clickable image, or a combination of both. Although HTML anchor elements contain a lot of functionality, they are still limiting—they require the use of the anchor element (<a>) itself, and they basically sit there waiting for someone to click them before navigating to the specified location.

The **XML Linking Language**, XLink, addresses and overcomes these limitations by allowing a link to another document to be specified on any element within an XML document. The links to other documents can be much more complex than the simple links supported by the HTMLspecification.

Example for XLink

```
<?xml version="1.0" encoding="UTF-8"?>
<homepages xmlns:xlink="http://www.w3.org/1999/xlink">
  <homepage xlink:type="simple" xlink:href="http://www.w3schools.com"> Welcome </homepage>
  <homepage xlink:type="simple" xlink:href="http://www.w3.org"> to RGCET </homepage>
</homepages>
```

Output

Welcome to RGCET

XLink Attributes

The XML Linking Language creates a link to another resource through the use of attributes specified on elements, not through the actual elements themselves. The XML Linking Language specification supports the attributes listed in the below table

Attribute	Description
xlink:type	This attribute must be specified and indicates what type of XLink is represented or defined.
xlink:href	This attribute contains the information necessary to locate the desired resource.
xlink:role	This attribute describes the function of the link between the current resource and another.
xlink:arcrole	This attributes describes the function of the link between the current resource and another.
xlink:title	This attribute describes the meaning of the link between the resources.
xlink:show	This attribute indicates how the resource linked to should be displayed
xlink:actuate	This attribute specifies when to load the linked resource.
xlink:label	This attribute is used to identify a name for a target resource.
xlink:from	This attribute identifies the starting resource.
xlink:to	This attribute identifies the ending resource.

1. The **xlink:type** attribute must contain one of the following values:

- ✓ simple
- ✓ extended
- ✓ locator
- ✓ arc
- ✓ resource
- ✓ title
- ✓ none

- A value of **simple** creates a simple link between resources.
- Indicating a value of **extended** creates an extended link.
- A value of **locator** creates a link that points to another resource.
- A value of **arc** creates an arc with multiple resources and various traversal paths.
- A **resource** value creates a link to indicate a specific resource.
- A value of **title** creates a title link.
- By specifying a value of **none** for the xlink:type attribute, the parent element has no XLink meaning, and no other XLink-related content or attributes have any relationship to the element.

2. The **xlink:href** attribute supplies the location of the resource to link to. This attribute is a URI reference that can be used to find the desired resource.

3. The **xlink:role** attribute specifies the function of the link. However, cannot use this attribute with just any type of XLink. This attribute may only be used for the following XLink types:
   - ✓ extended
   - ✓ simple
   - ✓ locator
   - ✓ resource

4. The **xlink:arcrole** attribute may only be used with two types of XLinks
   - ✓ Arc
   - ✓ simple

5. The **xlink:title** attribute is completely optional and is provide the document, in a way, what a particular link is. If the xlink:title attribute is specified, it should contain a string describing the resource.

6. The **xlink:show** attribute is an optionally specified attribute for a link for the simple and arc XLink types and accept the following values:
   - ✓ new
   - ✓ replace
   - ✓ embed
   - ✓ other
   - ✓ none

- If a value of **new** is specified, the resource will be loaded into a new window.
- A value of **replace** indicates that the resource should be loaded into the current window.
- Indicating a value of **embed** will load the resource into the specified location and wrap the originating resource around it, as appropriate. This effect is similar to specifying an src attribute on a <img> tag in HTML.
- A value of **other** allows each application using XLinks to look for other indications within the XML document to determine what needs to be done.
- Specifying a value of **none** has essentially the same effect as specifying a value of other, with the exception that the application is not expected to look for other indications as to what to do to display the link.

7. The **xlink:actuate** attribute is used to indicate when the linked resource should be loaded. This attribute will accept the following values:
   - ✓ onLoad
   - ✓ onRequest
   - ✓ other
   - ✓ none

- A value of **onLoad** indicates that when the current resource is loading, the linked resource should be loaded as well.
- Specifying a value of **onRequest** means the linked document should only be loaded when some post-loading event triggers a message for traversal.
- A value of **other** indicates that the application should look for other indications as to what the desired behavior is.
- A value of **none** specifies that the application is free to handle the loading of the linked resource in whatever manner seems appropriate.

8. The **xlink:label** attribute is used to name resource and locator XLink types.
9. This value will end up being used as values within the **xlink:from** and **xlink:to** attributes to indicate the starting and ending resources for an arc XLink type.

13. Explain the types of XLinks in detail? **(6 MARKS)**

The XML Linking Language offers two major types of links:

1. Simple links
2. Extended links

Within XLink, a simple link is a convenient, by which to associate two resources. These resource - one local and one remote - are connected by an arc, always making a simple link an outbound link. An extended link associates any number of resources together. Furthermore, those resources may be both local and remote.

Simple Links:

A **simple link** combines the functionality provided by the different pieces available through an extended link together into a shorthand notation. A simple link consists of an xlink:type attribute with a value of simple and, optionally, an xlink:href attribute with a specified value.

A simple link may have any content, and even no content; it is up to the application to provide some means to generate a traversal request for the link. If no target resource is specified with the xlink:href attribute, the link is simply considered "dead" and will not be traversable.

Simple links play multiple roles in linking documents. For instance, the simple link, itself, acts as a resource XLink type for the local document. It is the combination of this functionality that shortens the XLink definition for a simple link.

However simple links are just that—simple. They link exactly two resources together: one local and one remote. Therefore, if something more complex must be handled, an extended link is necessary.

Extended Links:

Within the XML Linking Language, **extended links** give the ability to specify relationships between an unlimited number of resources, both local and remote. In addition, these links can involve multiple paths between the linked resources. Local resources are part of the actual extended link, whereas remote resources identify external resources to the link. An out-of-line link is created when there are no local resources at all for a link.

Example

Sample.xml Contains a Modified Version of the Names List

```
<People xmlns:xlink="http://www.w3.org/1999/xlink" xlink:type="extended" xlink:title="Phone book">
    <Person>
        <Name> Dillon Larsen </Name>
        <Address>
            <Street> 123 Jones Rd. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip>77380</Zip>
        </Address>
    </Person>

    <Person>
        <Name> Madi Larsen </Name>
        <Address>
            <Street> 456 Hickory Ln. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77069 </Zip>
        </Address>
    </Person>

    <Person>
        <Name> John Doe </Name>
        <Address>
            <Street> 214 Papes Way </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77301 </Zip>
        </Address>
    </Person>
```

```
<Person xlink:type=”resource” xlink:label=”John”>
        <Name> John Smith </Name>
        <Spouse xlink:type=”resource” xlink:label=”JohnSpouse”> Jane Smith
        </Spouse> <Address>
                <Street> 522 Springwood Dr. </Street>
                <City> Houston </City>
                <State> TX </State>
                <Zip> 77069 </Zip>
        </Address>
</Person>

<Person xlink:type=”resource” xlink:label=”Jane”>
        <Name>Jane Smith</Name>
        <Spouse xlink:type=”resource” xlink:label=”JaneSpouse”>John
        Smith</Spouse> <Address>
                <Street>522 Springwood Dr.</Street>
                <City>Houston</City>
                <State>TX</State>
                <Zip>77069</Zip>
                </Address>
        </Person>
<Marriage xlink:type=”arc” xlink:from=”JohnSpouse” xlink:to=”Jane” xlink:actuate=”onRequest”
xlink:show=”new”/>
<Marriage xlink:type=”arc” xlink:from=”JaneSpouse” xlink:to=”John”
xlink:actuate=”onRequest” xlink:show=”new”/>

</People>
```

Output

Dillon Larsen

123 Jones Rd. Houston TX 77380

Madi Larsen

456 Hickory Ln. Houston TX 77069

John Doe

214 Papes Way Houston TX 77301

John Smith Jane Smith

522 Springwood Dr. Houston TX 77069

Jane Smith John Smith

522 Springwood Dr. Houston TX 77069

14. Explain in detail about XPath and its expressions? **(11 MARKS)**

The **XML Path Language (XPath)** is a standard for creating expressions that can be used to find specific pieces of information within an XML document.

XPath expressions are used by both XSLT (for which XPath provides the core functionality) and XPointer to locate a set of nodes.

To understand how XPath works, it helps to imagine an XML document as a tree of nodes consisting of both elements and attributes. An XPath expression can then be considered a sort of roadmap that indicates the branches of the tree to follow and what limbs hold the information desired.

XPath expressions have the ability to locate nodes based on the nodes' type, name, or value or by the relationship of the nodes to other nodes within the XML document. In addition to being able to find nodes based on these criteria, an XPath expression can also return any of the following:

- ✓ A node set
- ✓ A Boolean value
- ✓ A string value
- ✓ A numeric value

XML documents are a hierarchical tree of nodes. There is a similarity between URLs and XPath expressions.

- URLs represent a navigation path of a hierarchical file system.
- XPath expressions represent a navigation path for a hierarchical tree of nodes.

Operators and Special Characters

XPath expressions are composed using a set of operators and special characters, each with its own meaning. The various operators and special characters used within the XML Path Language.

Operator and special characters	Description
/	Selects from the root node
//	Selects nodes in the document from the current node that match the selection
.	Selects the current node
..	Selects the parent of the current node
*	A wildcard character that selects all elements or attributes regardless of name
@	Selects an attribute
:	Namespace separator
()	Indicates a grouping within an XPath expression
[expression]	Indicates a filter expression
[n]	Indicates that the node with the specified index should be selected
+	Addition operator

-	Subtraction operator
Div	Division operator
*	Multiplication operator
Mod	Returns the remainder of a division operation

The above table only provides a list of operators and special characters that can be used within an XPath expression. However, the table does not indicate what the order of precedence is.

The priority for evaluating XPath expressions is as follows:

1. Grouping
2. Filters
3. Path operations

XPath Syntax:

The XML Path Language provides a declarative notation, termed a *pattern*, used to select the desired set of nodes from XML documents. Each pattern describes a set of matching nodes to select from a hierarchical XML document. Each pattern describes a "navigation" path to the desired set of nodes similar to the Uniform Resource Identifier (URI) syntax. However, instead of navigating a file system, the XML Path Language navigates a hierarchical tree of nodes within an XML document.

Each "query" of an XML document occurs from a particular starting node that defines the context for the query. The context for the query has a very large impact on the results. For instance, the pattern that locates a node from the root of an XML document will most likely be a very different pattern when looking for the same node from somewhere else in the hierarchy.

One possible result from performing an XPath query is a node set, or a collection of nodes matching specified search criteria. To receive these results, a "location path" is needed to locate the result nodes. These location paths select the resulting node set relative to the current context. A location path is, itself, made up of one or more location steps. Each step is further comprised of three pieces:

- ✓ An axis
- ✓ A node test
- ✓ A Predicate

Therefore, the basic syntax for an XPath expression would be something like this:

axis::node test[predicate]

Using this basic syntax and the XML document in operators table, we could locate all the <c> nodes by using the following XPath expression:

/a/b/child::*

Alternatively, we could issue the following abbreviated version of the preceding expression:

/a/b/c

All XPath expressions are dependent on the current context. The context is the current location within the tree of nodes. Therefore, if we're currently on the second <b> element within the XML document, select all the <c> elements contained within that <b> element by using the following XPath expression:

./c

This is what's known as a "relative" XPath expression.

(i) Axes

The axis portion of the location step identifies the hierarchical relationship for the desired nodes from the current context.

Axis	description
ancestor	Specifies that the query should locate the ancestors of the current context node, which includes the parent node, the parent's parent node, and ultimately the root node.
ancestor-or-self	Indicates that in addition to the ancestors of the current context node, the context node should also be included in the resulting node set.
attribute	Specifies that the attributes of the current context node are desired.
child	Specifies that the immediate children of the current context node are desired.
descendant	Specifies that in addition to the immediate children of the current context node, the children's children are also desired.
descendant-or-self	Indicates that in addition to the descendants of the current context node, the current context node is also desired.
following	Specifies that nodes in the same document as the current context node that appear after the current context node should be selected.
following-sibling	Specifies that all the following siblings of the current context node should be selected.
Namespace	Specifies that all the nodes within the same namespace as the current context node should be selected.
parent	Selects the parent of the current context node.
preceding	Selects the nodes within the document that appear before the current context node.
preceding-sibling	Selects the siblings of the current context node that appear before the current context node.
self	Selects the current context node.

(ii) Node Tests

The **node test** portion of a location step indicates the type of node desired for the results. Every axis has a principal node type: If an axis is an element, the principal node type is element; otherwise, it is the type of node the axis can contain. For instance, if the axis is attribute, the principal node type is attribute.

A node test may also contain a node name, or QName. In this case, a node with the specified name is sought, and if found, it's returned in the node set. However, the nodes selected in this manner must be the principal node type sought and have an expanded name equal to the QName specified. This means that if the node belongs to a namespace, the namespace must also be included in the node test for the node to be selected.

For instance, ancestor::div and ancestor::test:div will produce two entirely different node sets. In this first case, only nodes that have no namespace specified and have a name of div will be selected. In the second case, only those div nodes belonging to the test namespace will be selected.

In addition to specifying an actual node name, other node tests are available to select the desired nodes. The list of these node tests:

- comment()
- node()
- processing-instruction()
- text()

The small number of node tests are available for use within a location step.

- ✓ The **comment()** node test selects comment nodes from an XML document.
- ✓ The **node()** node test selects a node of any type
- ✓ The **text()** node test selects those nodes that are text nodes.
- ✓ The **processing-instruction()** node test, because this node test will accept a literal string parameter to specify the name of a desired processing instruction.

(iii) Predicates

The **predicate** portion of a location step filters a node set on the specified axis to create a new node set. Each node in the preliminary node set is evaluated against the predicate to see whether it matches the filter criteria. If it does, the node ends up in the filtered node set. Otherwise, it doesn't.

A predicate may consist of a filter condition that is applied to an axis that either directs the condition in a forward or reverse direction.

- A **forward axis predicate** contains the current context node and nodes that follow the context node.
- A **reverse axis predicate** contains the current context node and nodes that precede the context node.

15. Explain in detail about XPath Functions? **(6 MARKS)**

XPath functions are used to evaluate XPath expressions and can be divided into one of four main groups:

- Boolean
- Nodeset
- Number
- String

Type	Syntax	Return Type	Description
Boolean	boolean(object)	Boolean	Converts the argument into a Boolean value
	false()	Boolean	Returns False
	lang(string)	Boolean	Returns True if the xml:lang attribute of the context node is the same as the sub language specified by the argument
	not(expression)	Boolean	Returns True if the expression argument is False
	true()	Boolean	Returns True
Node set	count(node-set)	Number	Returns the number of nodes in the node set argument
	Document(variant, [node-set])	Node set	Creates an XML document from the variant argument
	id(object)	Node set	Returns a node set based on the node's unique ID
	key(name, value)	Node set	Returns a node set with the specified key name and value
	last()	Number	Returns the context size for the expression evaluation context
	local-name ([node-set])	String	Returns the local part of the expanded name of the first node in the node set
	name([node-set])	String	Returns the expanded name of the first node in the node set
	namespace-uri ([node-set])	String	Returns the name- space URI for the namespace of the first node in the node set
	position()	Number	Returns the index number of the node within the parent
Number	ceiling(number)	Number	Returns the smallest integer that is not less than the argument
	floor(number)	Number	Returns the largest integer that is not greater than the argument
	number(variant)	Number	Converts the argument to a number
	round(number)	Number	Returns the integer closest in value to the argument

	sum(node-set)	Number	Sums the value of all the nodes within the node set after being converted to a number
String	concat(string, constring,[string*])	String	Returns the concatenation of the string arguments
	contains(string, string)	Boolean	Returns True if the first string contains the second
	normalize- space()	String	Returns the string with the whitespace removed
	starts-with(string, string)	Boolean	Returns True if the first string begins with the second string
	string(variant)	String	Converts the argument into a string
	string-length (string)	Number	Returns the length of the string
	substring(string, start, length)	String	Returns a substring of the specified string starting at the start position indicated by startpos of the length specified by length
	substring-after (string, string)	String	Returns the substring of the first string that follows the first occurrence of the second string
	substring-before(chars, replace)	String	Returns the substring of the first string that precedes the first occurrence of the second string
	translate(string, chars, replace)	String	Replaces the string specified in chars within the specified string with the string specified in replace.

16. Explain in detail about XPointer? **(11 MARKS)**

- XPointer allows links to point to specific parts of an XML document.
- XPointer uses XPath expressions to navigate in the XML document.
- XPointer describes a location within an external document.
- XPointer can target a point within that XML document or a range within the target XML document.

The **XML Pointer Language (XPointer)** describes a location within an external document, an XPointer can target a point within that XML document or a range within the target XML document. XPointer builds on the XPath specification, the location steps within an XPointer are comprised of the same elements that make up XPath location steps.

XPointer provides two more important node tests:

- ✓ point()
- ✓ range()

The XPointer specification added the concept of a location within an XML document. Within XPointer, a location can be an XPath node, a point, or a range.

A **point** can represent the location immediately before or after a specified character or the location just before or just after a specified node.

A **range** consists of a start point and an endpoint and contains all XML information between those two points. In fact, the XPointer specification extends the node types to include points and ranges. XPointer expressions also allow predicates to be specified as part of a location step in much the same fashion XPath expressions allow for them. As with XPath expressions, XPointer expressions have specific functions to deal with each specific predicate type.

However, the XPointer specification also adds an additional function named unique().This new function indicates whether an XPointer expression selects a single location rather than multiple locations or no locations at all.

For an XPath expression, the result from a location step is known as a ***node set***; for an XPointer expression, the result is known as a ***location set***. To reduce the confusion, the XPointer specification uses a different term for the results of an expression:

Because an XPointer expression can yield a result consisting of points or ranges, the idea of the *node set* had to be extended to include these types. Therefore, to prevent confusion, the results of an XPointer expression are referred to *location sets*.

Four of the functions that return location sets, id(), root(), here(), and origin(),

Some XPointer Functions That Return Location Sets

Function	description
id()	Selects all nodes with the specified ID
root()	Selects the root element as the only location in a location set
here()	Selects the current element location in a location set
origin()	Selects the current element location for a node using an out-of-line link

- The id() function works exactly the same as the id() function for an XPath expression.
- The root() function works just like the / character—it indicates the root element of an XML document.
- The here() function, as indicated, refers to the current element. Because an XPointer expression can be located in a text node or in an attribute value, this function could be used to refer to the current element rather than simply the current node.
- The origin() function works much the same as the here() function, except that it refers to the originating element. The key idea here is that the originating element does not need to be located within the same document as the resulting location set. Not every target for an XPointer must be a node.

Points

Many times a link from one XML document into another must locate a specific point within the target document.

Two different types of points can be represented using XPointer points:

- ✓ Node points
- ✓ Character points

Node points are location points in an XML document that are nodes that contain child nodes. For these node points, the index position indicates after which child node to navigate to. If 0 is specified for the index, the point is considered to be immediately before any child nodes. A node point could be considered to be the gap between the child nodes of a container node.

When the origin node is a text node, the index position indicates the number of characters. These location points are referred to as ***character points*.** Because you are indicating the number of characters from the origin, the index specified must be an integer greater than or equal to 0 and less than or equal to the total length of the text within the text node. By specifying 0 for the index position in a character point, the point is considered to be immediately before the first character in the text string. For a character point, the point, conceptually, represents the space between the characters of a text string.

Example:

```
<People>
    <Person>
        <Name> Dillon Larsen </Name>
        <Address>
            <Street> 123 Jones Rd. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77380 </Zip>
        </Address>
    </Person>

    <Person>
        <Name> Madi Larsen </Name>
        <Address>
            <Street> 456 Hickory Ln. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77069 </Zip>
        </Address>
    </Person>
```

```
    <Person>
        <Name> John Doe </Name>
        <Address>
            <Street> 214 Papes Way </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77301 </Zip>
            </Address>
    </Person>

    <Person>
        <Name> John Smith </Name>
        <Address>
            <Street> 522 Springwood Dr. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77069 </Zip>
        </Address>
    </Person>

    <Person>
        <Name> Jane Smith </Name>
        <Address>
            <Street> 522 Springwood Dr. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77069 </Zip>
        </Address>
    </Person>

    <Person>
        <Name> Mark Boudreaux </Name>
        <Address>
            <Street> 623 Fell St. </Street>
            <City> Houston </City>
            <State> TX </State>
            <Zip> 77380 </Zip>
        </Address>
    </Person>
</People>
```

Result

Dillon Larsen
123 Jones Rd. Houston TX 77380
Madi Larsen
456 Hickory Ln. Houston TX 77069
John Doe
214 Papes Way Houston TX 77301
John Smith
522 Springwood Dr. Houston TX 77069
Jane Smith
522 Springwood Dr. Houston TX 77069
Mark Boudreaux
623 Fell St. Houston TX 77380

Ranges

An XPointer range defines just that—a range consisting of a start point and an endpoint. A range will contain the XML between the start point and endpoint but does not necessarily have to consist of neat sub trees of an XML document. A range can extend over multiple branches of an XML document. The only criterion is that the start point and endpoint must be valid. Within the XPointer Language, a range can be specified by using the keyword to within the XPointer expression in conjunction with the start-point() and end-point() functions.

For instance, the following expression specifies a range beginning at the first character in the <Name> element for Dillon Larsen and ending after the ninth character in the <Name> element for Dillon Larsen:

/People/Person[1]/Name/text()start-point()[position()=0] to
/People/Person[1]/Name/text()start-point()[position()=9]

In this example, two node points are used as the starting and ending points for the range. The result is the string Dillon La.

XPointer Range Functions

Function	Description
end-point()	Selects a location set consisting of the endpoints of the desired location steps
range-inside()	Selects the range(s) covering each location in the location-set argument
range-to()	Selects a range that completely covers the locations within the location-set argument
start-point()	Selects a location set consisting of the start points of the desired location steps

The XML Pointer Language also has the ability to perform basic string matching by using a function named string-range(). This function returns a location set with one range for every non-overlapping match to the search string by performing a case-sensitive search.

The general syntax for string-range() is as follows:

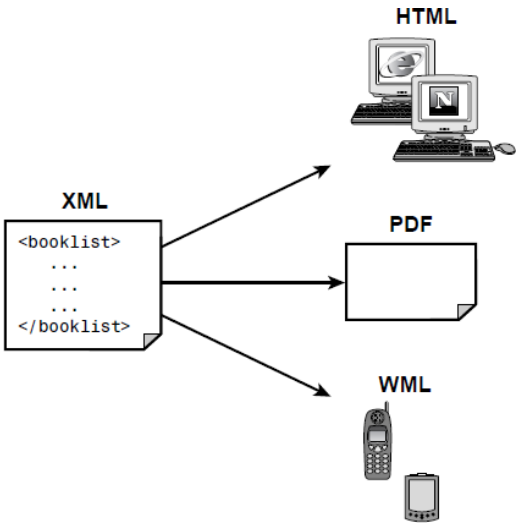
string-range(location-set, string, [index, [length]])

The location-set argument for the string-range() function is any XPointer expression that would create a location set as its result—for instance, /, /People/Person, /People/Person[1], and so on. The string argument contains the string searched for. It does not matter, when you're using the string-range() function, where this string occurs; only that is does occur. By specifying the index and length arguments, you can indicate the range you wish returned. For instance, to return the letters *Ma* from the Madi Larsen <Name> element, you could pass an index value of 1 and a length value of 2.

17. Explain the XSLT for Document Publishing in detail? (11 MARKS) (NOV 2017)

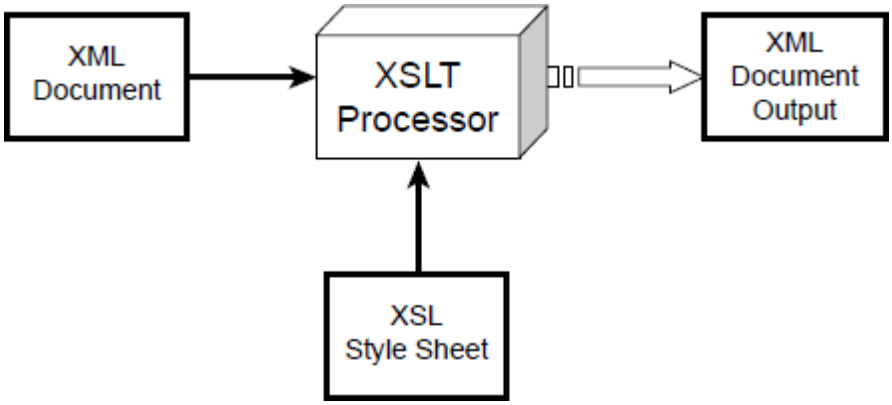
XML Stylesheet Language Transformation (XSLT) has an important role in the field of document publishing. Imagine, for example, that we have an XML document for a list of books. To publish this document in various formats. Using XSL, we can convert the book list to an HTML file, PDF document, or other format. The key to this example is the XML document, which serves as a single data source. By applying an XSL style sheet, we render a new view of the data. The development of multiple style sheets allows us to have multiple views of the same data. This approach provides a separation of the data (the XML document) and the view (the XSL style sheet).

The example to support wireless Internet clients. A growing number of mobile phones and PDAs support the Wireless Application Protocol (WAP). These WAP enabled devices contain a mini browser for rendering Wireless Markup Language (WML) documents. To support the wireless Internet clients, all we have to do is design an appropriate XSL style sheet to convert the XML document to WML. No modifications are required to the original XML document.



Publishing documents with XSLT

XSLT provides the mechanism for converting an XML document to another format. This is accomplished by applying an XSLT style sheet to the XML document. The style sheet contains conversion rules for accessing and transforming the input XML document to a different output format. An XSLT processor is responsible for applying the rules defined in the style sheet to the input XML document.



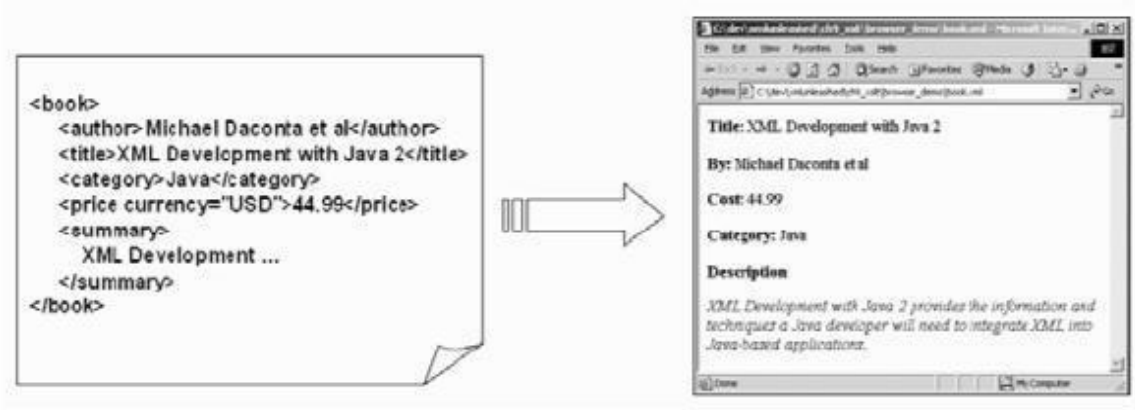
Sending data to the XSLT processor

Example

Convert an XML document to an HTML document. The XML document contains a list of books, as shown below:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="book_view.xsl"?>
<book>
    <author> Michael Daconta et al </author>
    <title> XML Development with Java 2 </title>
    <category> Java </category>
    <price currency="USD"> 44.99 </price>
    <summary>
            XML Development with Java 2 provides the
            information and techniques a Java developer will need
            to integrate XML into Java-based applications.
    </summary>
</book>
```

The desired output of the HTML table



Creating the XML Document

The XML document, book.xml, contains elements for the author, title, price, summary, and category. Below listing has the complete code for **book.xml**. <?xml version="1.0"?>

```
<book>
   <author> Michael Daconta et al </author>
   <title> XML Development with Java 2 </title>
   <category> Java </category>
   <price currency="USD"> 44.99 </price>
   <summary>
         XML Development with Java 2 provides the
         information and techniques a Java developer will need
         to integrate XML into Java-based applications.
   </summary>
</book>
```

Creating the XSL Style Sheet

The next step is to create the XSL style sheet. XSL style sheets are XML documents; as a result, they must be well formed. An XSL style sheet has the following general structure:

```
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="URI" version="1.0">
      <!--XSL-T CONVERSION RULES-->
</xsl:stylesheet>
```

The <xsl:stylesheet> element defines how the XSLT processor should process the current XSL document. The xmlns attribute is the namespace definition. The XSL Transformation engine reads the xmlns attribute and determines whether it supports the given namespace. The xmlns attribute specifies the XSL prefix. All XSL elements and types in the document use the prefix.

The xmlns attribute value contains a Uniform Resource Identifier (URI), which serves as a generic method for identifying entities on the World Wide Web.

The XSL style sheet contains HTML text and XSL elements. The HTML text forms the basis of the desired output page. The XSL elements are template rules for the XSLT processor. A template is associated with a given element in the XML document. In our example, a template is defined to match on the <book> element using the following code:

```
<xsl:template match="/book">
      <!--static text and xsl rules -->
</xsl:template>
```

XSLT defines the <xsl:value-of> element for retrieving data from a XML document. The <xsl:value-of> element contains a select attribute. This attribute value is the name of the actual XML element you want to retrieve. For example, the following code will retrieve the title of the book:

<xsl:value-of select="title" />

To create the file book_view.xsl. This style sheet will create an HTML page that contains information about the book, which is stored in the file book.xml. The style sheet contains the basic format of an HTML page and uses XSL elements to retrieve the data. Currently, the XSL elements are merely placeholders in the document. Once an XSL processor accesses the XSL style sheet, the processor executes the XSL elements and replaces them with the appropriate data from the XML document. Below code contains the complete code for book_view.xsl.

Example

```
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:template match="/book">
   <html>
         <body>
               <b> Title: </b> <xsl:value-of select="title" />
               <p/>
               <b> By: </b> <xsl:value-of select="author" />
               <p/>
               <b> Cost: </b> <xsl:value-of select="price" />
               <p/>
               <b> Category: </b> <xsl:value-of select="category"
               /> <p/>
               <b> Description </b>
               <p/>
               <i> <xsl:value-of select="summary" /> </i>
         </body>
   </html>
</xsl:template>
</xsl:stylesheet>
```

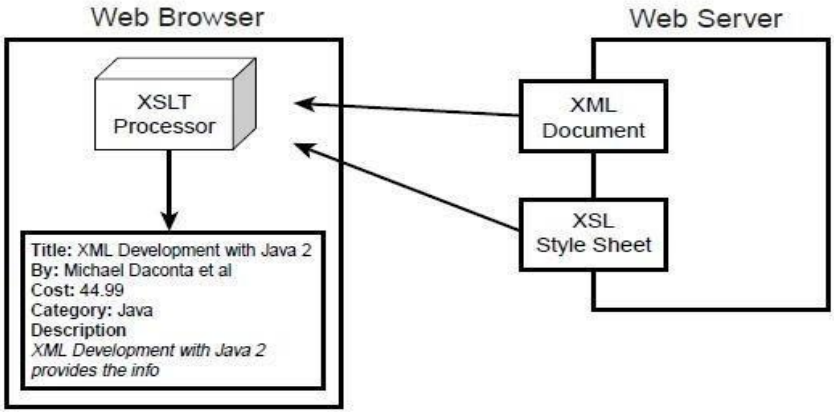
The XSLT Processor

Two techniques are available for performing the XSLT processing:

1. Client-side processing
2. Server-side processing

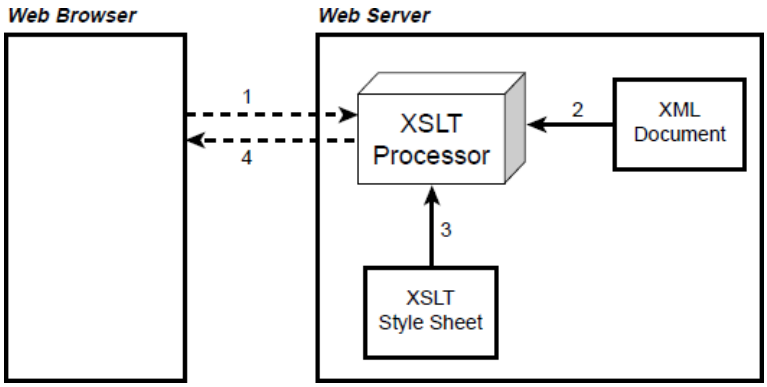
Client-side XSLT processing commonly occurs in a Web browser. The Web browser includes an XSLT processor and retrieves the XML document and XSL style sheet.

The client-side technique offloads the XSLT processing to the client machine. This minimizes the workload on the Web server. However, the disadvantage is that the Web browser must provide XSLT support. Netscape Communicator 6 and Microsoft Internet Explorer 6 support the XSLT 1.0 specification.



Processing XSLT in a Web browser

Server-side XSLT processing occurs on the Web server or application server. A server-side process such as an Active Server Page (ASP), Java Server Page (JSP), or Java servlet will retrieve the XML document and XSL style sheet and pass them to an XSLT processor. The output of the XSLT processor is sent to the client Web browser for presentation. The output is generally a markup language, such as HTML, that is understood by the client browser. The application interaction is shown below figure.



Processing XSLT on the server side

An advantage of the server-side technique is browser independence. The output document is simply an HTML file. This technique supports the older browser versions and makes the application more robust and versatile.

The server-side technique, the application can support a diverse collection of clients. The application can detect the user-agent, such as a WAP-enabled mobile phone, and send back a document containing a Wireless Markup Language (WML) tag. The WAP-enabled phone can render the content using the built-in WML mini-browser.

18. Explain in detail about XSL Formatting Objects? (11 MARKS)

- The XSL technology is also composed of **XSL Formatting Objects (XSL-FO).**
- XSL-FO was designed to assist with the printing and displaying of XML data. The main emphasis is on the document layout and structure. This includes the dimensions of the output document, including page headers, footers, and margins.
- XSL-FO also allows the developer to define the formatting rules for the content, such as font, style, color, and positioning.XSL-FO is a sophisticated version of Cascading Style Sheets (CSS). XSL-FO borrows a lot of the terminology and elements from CSS.
- XSL-FO documents are well-formed XML documents. An XSL-FO formatting engine processes XSL-FO documents.

The two techniques for creating XSL-FO documents.

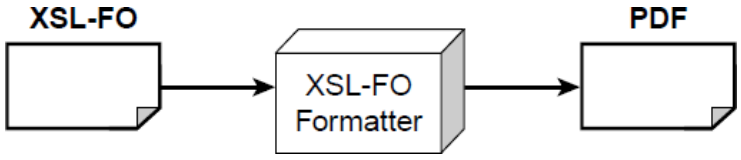
- ✓ The first is to simply develop the XSL-FO file with the included data.
- ✓ The second technique is to dynamically create the XSL-FO file using an XSLT translation.

XSL-FO Formatting Engines

Many of the XSL-FO formatting engines implement a subset of the XSL-FO specification. Also, the browser support for XSL-FO is nonexistent. Engines are available that allow you to experiment with the basic features of XSL-FO. To use the Apache XSL-FOP to generate PDF documents from XML.

XSL-FO Engine	Web Site
Apache XSL-FOP	xml.apache.org
XEP	www.renderx.com
iText	www.lowagie.com/iText/
Unicorn	www.unicorn-enterprises.com

To create a simple XSL-FO document. Once the document is created, we will use the Apache XSL-FOP formatter to convert the document to a PDF file. The application interaction is illustrated below figure.



Apache XSL-FOP generating PDF documents

Basic Document Structure

An XML-FO document follows the syntax rules of XML; as a result, it is well formed.

XSL-FO elements use the following namespace:

http://www.w3.org/1999/XSL/Format

The following code snippet shows the basic document setup for XSL-FO:

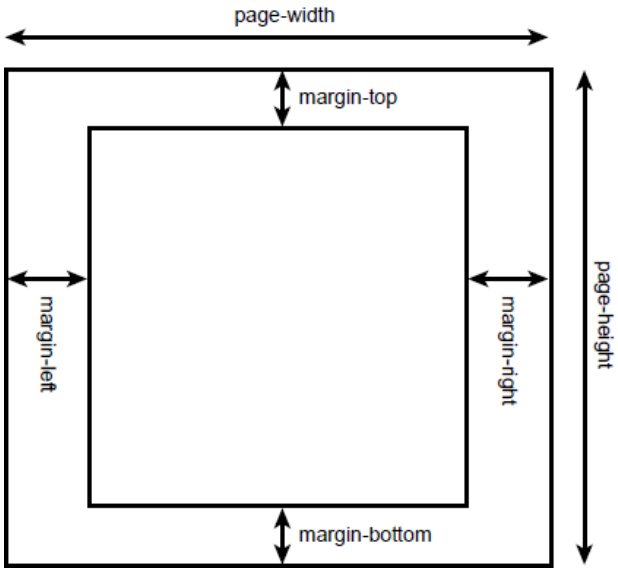
```
<?xml version="1.0" encoding="utf-8"?>
  <fo:root xmlns:fo="http://www.w3.org/1999/XSL/Format">
    <!-- layout master set -->
     <!-- page masters: size and layout -->
    <!-- page sequences and content -->
  </fo:root>
```

The **element <fo:root>** is the root element for the XSL-FO document. An XSL-FO document can contain the following components:

1. Page master
2. Page master set
3. Page sequences

(i) Page Master: <fo:page-master>

The **page master** describes the page size and layout. For example, we could use an 8.5×11-inch page or an A4 letter. The page master contains the dimensions for a page, including width, height, and margins. The page master is similar to a slide master in Microsoft PowerPoint. The components of the page master are shown.



Components of the page master

The <fo:simple-page-master> element defines the layout of a page. The following code snippet describes a U.S. letter:

```
<fo:simple-page-master master-name=”simple”
        page-height=”11in”
        page-width=”8.5in”
        margin-top=”1in”
        margin-bottom=”1in”
        margin-left=”1.25in”
        margin-right=”1.25in”>
</fo:simple-page-master>
```

The attributes for <fo:simple-page-master>. The attributes define the height and width of the page, along with the size of the margins. The dimensions in this example are listed in inches (in).

XSL-FO Dimensions

Unit Suffix	Description
in	Inches (1 inch equals 2.54 centimeters)
mm	Millimeters
cm	Centimeters
pt	Points (1 point equals 1/72 inch)
pc	Picas (1 pica equals 12 points)
em	Font size of the relevant font
ex	X-height of the relevant font
px	Pixels

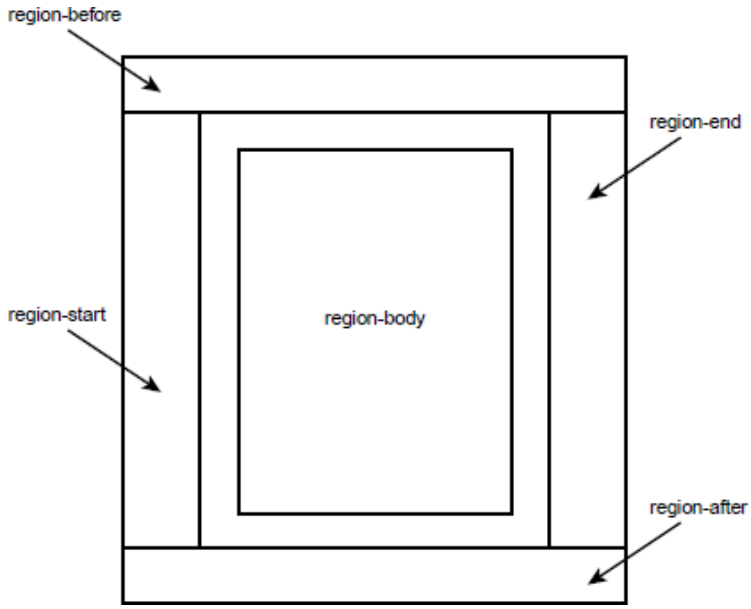
To set the page height to 210 millimeters, use the following syntax:

```
page-height=”210mm”
```

The <fo:simple-page-master> element can also be used to describe an A4 letter (height 210 mm and width 297 mm):

```
<fo:simple-page-master master-name=”A4-example”
    page-height=”210mm”
    page-width=”297mm”
    margin-top=”0.5in”
    margin-bottom=”0.5in”
    margin-left=”0.5in”
    margin-right=”0.5in”>
</fo:simple-page-master>
```

Each page is divided into five *regions.* Regions serve as containers for the document content. The regions are depicted below in the below Figure.



The region-before and region-after areas are commonly used for page headers and footers. The region-body area is the center of the page and contains the main content. The region-start and region-end sections are commonly used for left and right sidebars, respectively. During the definition of a page master, you specify the size of the regions using the following elements:

i. <fo:region-before>
ii. <fo:region-after>
iii. <fo:region-body>
iv. <fo:region-start>
v. <fo:region-end>

The following example defines the dimensions for <fo:region-body>, <fo:regionbefore>, and <fo:region-after>:

```
<fo:simple-page-master master-name=”simple”
                       page-height=”11in”
                       page-width=”8.5in”>
    <fo:region-body margin-top=”0.5in”/>
    <fo:region-before extent=”0.5in”/>
    <fo:region-after extent=”0.5in”/>
</fo:simple-page-master>
```

The extent attribute has a different meaning, depending on the region. For <fo:regionend> and <fo:region-start>, the extent attribute specifies the width. For <fo:regionbefore> and <fo:region-after>, it specifies the height.

(ii) Page Master Set: <fo:page-master-set>

A document can be composed of multiple pages, each with its own dimensions. **The page master set** refers to the collection of page masters.

In the following code example, a page master set is defined that contains one page set:

<fo:layout-master-set>

```
<fo:simple-page-master master-name="simple"
                       page-height="11in"
                       page-width="8.5in"
                       margin-top="1in"
                       margin-bottom="1in"
                       margin-left="1.25in"
                       margin-right="1.25in">
    <fo:region-body margin-top="0.5in"/>
    <fo:region-before extent="3cm"/>
    <fo:region-after extent="1.5cm"/>
</fo:simple-page-master>
</fo:layout-master-set>
```

(iii) Page Sequences: <fo:page-sequence>

A **page sequence** defines a series of printed pages. Each page sequence refers to a page master for its dimensions. The page sequence contains the actual content for the document.

- The <fo:page-sequence> element contains <fo:static-content> and <fo:flow> elements.
- The <fo:static-content> element is used for page headers and footers. For example,we can define a header for the company name and page number, and this information will appear on every page.
- The <fo:flow> element contains a collection of text blocks. The <fo:flow> element is similar to a collection of paragraphs. A body of text is defined using the <fo:block> element.The <fo:block> element is a child element of <fo:flow>. The <fo:block> element contains free-flowing text that will wrap to the next line in a document if it overflows.

Page Headers and Footers

The <fo:static-content> element defines content that should appear on every page.The <fo:static-content> element is commonly used to set up page headers and footers. The <fo:static-content> element is a component of <fo:page-sequence>.

A **page header** that contains the company name and current page number. We'll also define a footer that lists the company's Web site. This example is also composed of multiple pages to illustrate the fact that the header and footer are repeated on each page.

The header is defined using the following code fragment:

```
<!-- header -->
<fo:static-content flow-name="xsl-region-before">
        <fo:block text-align="end"
                font-size="10pt"
                font-family="serif"
                line-height="14pt" >
            Ez Books Catalog - page <fo:page-number/>
        </fo:block>
</fo:static-content>
```

The content for the header is placed in xsl-region-before, which is the top of the page in this example. The <fo:block> element uses the text-align attribute to place the text at the end of the region. This example uses the English language, so the text is right justified. The current page number is determined using the <fo:page-number> element.

The footer is defined using the following code fragment:

```
<!-- footer -->
<fo:static-content flow-name="xsl-region-after">
    <fo:block text-align="center"
            font-size="10pt"
            font-family="serif"
            line-height="14pt" >
        Visit our website http://www.ezbooks.web
    </fo:block>
</fo:static-content>
```

The **footer content** is placed at the bottom of the page in xsl-region-after. A message containing the company's Web site is listed in the footer.

Graphics

XSL-FO also allows for the insertion of external graphic images. The graphic formats supported are dependent on the XSL-FO formatting engine. The Apache-FOP formatting engine supports the popular graphics formats: GIF, JPEG, and BMP.

The following code fragment inserts the image smiley.jpg:

```
<fo:block text-align="center">
        <fo:external-graphic src="smiley.jpg" width="200px"
height="200px"/> </fo:block>
```

Tables

XSL-FO has rich support for structuring tabular data. In fact, there are many similarities between HTML tables and XSL-FO tables.

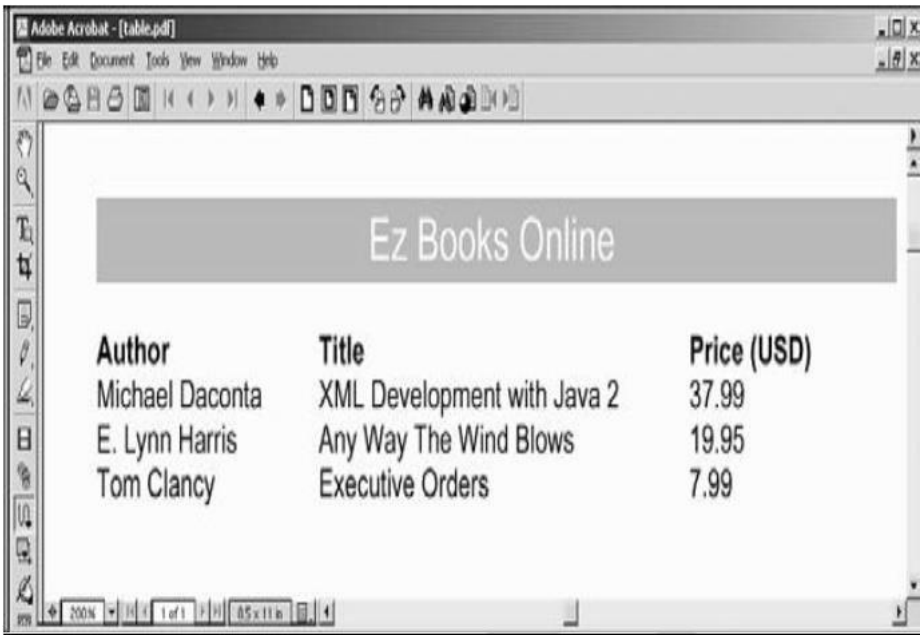
Comparing HTML Table Elements and XSL-FO Table Elements

HTML Element	XSL-FO Element
TABLE	fo:table-and-caption
Not applicable	fo:table
CAPTION	fo:table-caption
COL	fo:table-column
COLGROUP	Not applicable
TH	fo:table-header
TBODY	fo:table-body
TFOOT	fo:table-footer
TD	fo:table-cell
TR	fo:table-row

The following code fragment defines the basic structure of the table:

```
<fo:table>
   <!-- define column widths -->
   <fo:table-column column-width=”120pt”/>
   <fo:table-column column-width=”200pt”/>
   <fo:table-column column-width=”80pt”/>
    <fo:table-header>
   <fo:table-row>
            <fo:table-cell>
                  <fo:block font-weight=”bold”>Author</fo:block>
            </fo:table-cell>
            <fo:table-cell>
                  <fo:block font-weight=”bold”>Title</fo:block>
            </fo:table-cell>
            <fo:table-cell>
                  <fo:block font-weight=”bold”>Price
            (USD)</fo:block> </fo:table-cell>
   </fo:table-row>
    </fo:table-header>
   <!-- insert table body and rows here -->
</fo:table>
```

Output



19. Explain in detail about Parsing XML using DOM? (11 MARKS) (APR 2017)

- The **Document Object Model (DOM)** provides a way of representing an XML document in memory so that it can be manipulated by your software.
- DOM is a standard application programming interface (API) that makes it easy for programmers to access elements and delete, add, or edit content and attributes.
- DOM was proposed by the World Wide Web Consortium (W3C) in August of 1997 in the User Interface Domain. The Activity was eventually moved to the Architecture Domain in November of 2000.
- DOM interfaces are defined independent of any particular programming language.
- DOM code in just about any programming language, such as Java, ECMAScript (a standardized version of JavaScript/JScript), or C++.

The XML DOM is:

- A standard object model for XML
- A standard programming interface for XML
- Platform- and language-independent
  - ✓ The XML DOM defines the **objects and properties** of all XML elements, and the **methods** (interface) to access them.
  - ✓ In other words: **The XML DOM is a standard for how to get, change, add, or delete XML elements.**

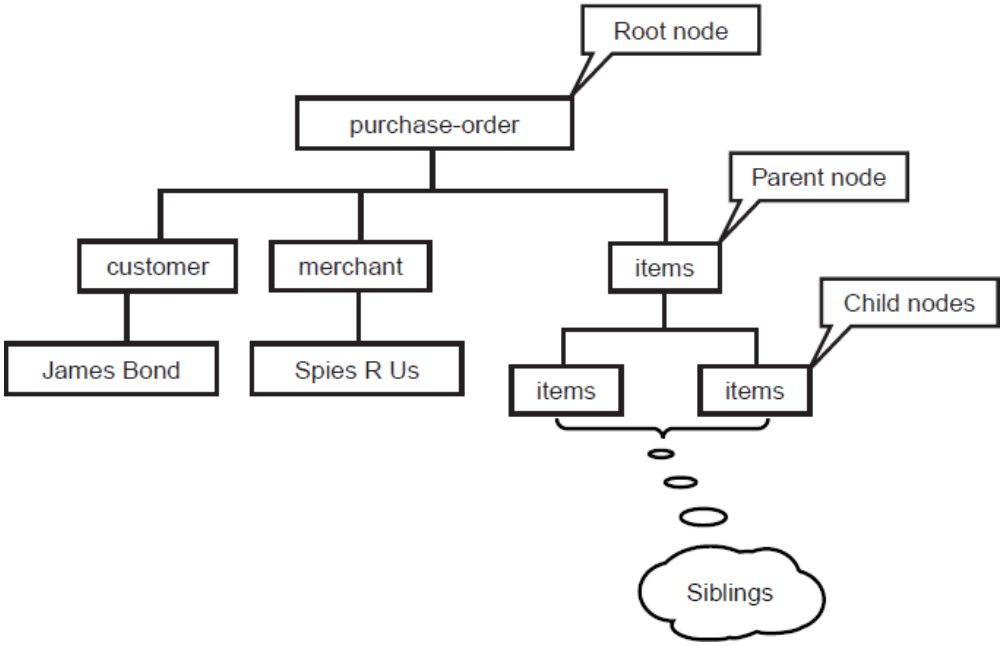
DOM Core

- DOM is a tree structure that represents elements, attributes, and content.

Consider a simple XML document

```
<purchase-order>
        <customer> James Bond </customer>
        <merchant> Spies R Us </merchant>
        <items>
                <item> Night vision camera </item>
                <item> Vibrating massager </item>
        </items>
</purchase-order>
```

Tree structure representing the XML document as



The DOM says:

- The entire document is a document node
- Every XML element is an element node
- The text in the XML elements are text nodes
- Every attribute is an attribute node
- Comments are comment nodes

Parents, Children, and Siblings

The nodes in the node tree have a hierarchical relationship to each other.

The terms parent, child, and sibling are used to describe the relationships. Parent nodes have children. Children on the same level are called siblings (brothers or sisters).

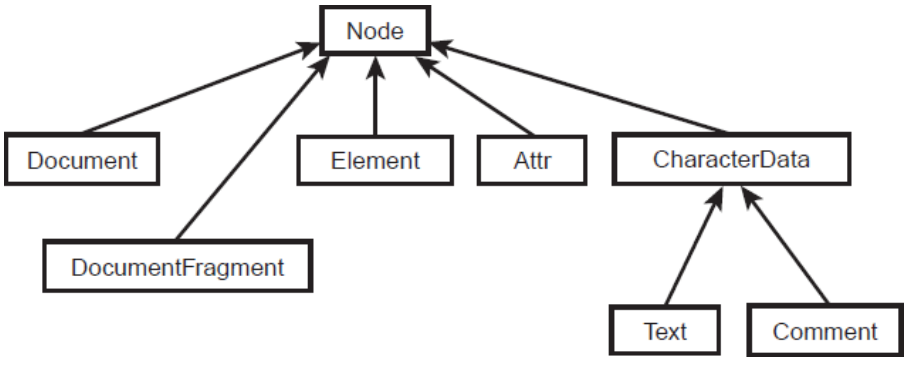
- ✓ In a node tree, the top node is called the root
- ✓ Every node, except the root, has exactly one parent node
- ✓ A node can have any number of children
- ✓ A leaf is a node with no children
- ✓ Siblings are nodes with the same parent

DOM Interfaces

DOM interfaces are defined in IDL so that they are language neutral.

Interface	Description
Node	The primary interface for the DOM. It can be an element, attribute, text, and so on, and contains methods for traversing a DOM tree.
NodeList	An ordered collection of Nodes.
NamedNodeMap	An unordered collection of Nodes that can be accessed by name and used with attributes.
Document	An Node representing an entire document. It contains the root Node.
DocumentFragment	A Node representing a piece of a document. It's useful for extracting or inserting a fragment into a document.
Element	A Node representing an XML element.
Attr	A Node representing an XML attribute.
CharacterData	A Node representing character data.
Text	A CharacterData node representing text.
Comment	A CharacterData node representing a comment.
DOMException	An exception raised upon failure of an operation.
DOMImplementation	Methods for creating documents and determining whether an implementation has certain features.

The relationships among the interfaces



Extended Interfaces

Interface	Description
CDATASection	Text representing CDATA
DocumentType	A node representing document type
Notation	A node with public and system IDs of a notation
Entity	A node representing an entity that's either parsed or unparsed
EntityReference	A node representing an entity reference
ProcessingInstruction	A node representing an XML processing instruction

DOM Traversal and Range

Traversal is a convenient way to walk through a DOM tree and select specific nodes.

Traversal Interfaces

Interface	Description
NodeIterator	Used to walk through nodes linearly. Represents a subtree as a linear list
TreeWalker	Represents a subtree as a tree view
NodeFilter	Can be used in conjunction with NodeIterator and TreeWalker to select specific nodes.
DocumentTraversal	Contains methods to create NodeIterator and TreeWalker instances

Range

Range interfaces provide a convenient way to select, delete, extract, and insert content.

A range consists of two boundary points corresponding to the start and the end of the range. A boundary point's position in a Document or DocumentFragment tree can be characterized by a node and an offset. The node is the container of the boundary point and its position. The container and its ancestors are the ancestor containers of the boundary point and its position. The offset within the node is the offset of the boundary point and its position. If the container is an Attr, Document, DocumentFragment, Element, or EntityReference node, the offset is between its child nodes. If the container is a CharacterData, Comment, or ProcessingInstruction node, the offset is between the 16-bit units of the UTF-16 encoded string contained by it.

The boundary points of a range must have a common ancestor container that is either a Document, DocumentFragment, or Attr node. That is, the content of a range must be entirely within the subtree rooted by a single Document, DocumentFragment, or Attr node. This common ancestor container is known as the *root container* of the range.

The tree rooted by the root container is known as the range's *context tree*.

The container of a boundary point of a range must be an Element, Comment, ProcessingInstruction, EntityReference, CDATASection, Document, DocumentFragment, Attr, or Text node. None of the ancestor containers of the boundary point of a range can be a DocumentType, Entity, or Notation node.

Range Interfaces

Interface	Description
Range	This interface describes a range and contains methods to define, delete, and insert content.
DocumentRange	This interface creates a range.

Steps to Using DOM

Following are the steps used while parsing a document using DOM Parser.

1. Import XML-related packages.
2. Create a DocumentBuilder
3. Create a Document from a file or stream
4. Extract the root element
5. Examine attributes
6. Examine sub-elements

Import XML-related packages

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import java.io.*;
```

Create a DocumentBuilder

```
DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
```

Create a Document from a file or stream

```
StringBuilder xmlStringBuilder = new StringBuilder(); xmlStringBuilder.append("<?xml
version="1.0"?> <class> </class>"); ByteArrayInputStream input = new
ByteArrayInputStream(xmlStringBuilder.toString(). getBytes("UTF-8"));

Document doc = builder.parse(input);
```

Extract the root element

```
Element root = document.getDocumentElement();
```

Examine attributes

```
getAttribute("attributeName");    //returns specific attribute
getAttributes();                  //returns a Map (table) of names/values
```

Examine sub-elements

```
getElementsByTagName("subelementName");    //returns a list of subelements of specified name
getChildNodes();                  //returns a list of all child nodes
```

Example:

Here is the input.xml file we need to parse:

```
<?xml version="1.0"?>
   <class>
         <student rollno="393">
               <firstname> dinkar </firstname>
               <lastname> kad </lastname>
               <nickname> dinkar </nickname>
               <marks> 85 </marks>
         </student>

         <student rollno="493">
               <firstname> Vaneet </firstname>
               <lastname> Gupta </lastname>
               <nickname> vinni </nickname>
               <marks> 95 </marks>
         </student>

         <student rollno="593">
               <firstname> jasvir </firstname>
               <lastname> singn </lastname>
               <nickname> jazz </nickname>
               <marks> 90 </marks>
         </student>
</class>
```

DomParserDemo.java:

```
import java.io.File;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;
import org.w3c.dom.Node;
import org.w3c.dom.Element;
public class DomParserDemo {
  public static void main(String[] args){
    try {
      File inputFile = new File("input.txt");
      DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
      DocumentBuilder dBuilder = dbFactory.newDocumentBuilder(); Document doc
      = dBuilder.parse(inputFile);
      doc.getDocumentElement().normalize();
      System.out.println("Root element :" + doc.getDocumentElement().getNodeName());
      NodeList nList = doc.getElementsByTagName("student"); System.out.println("----------
      ----------");
      for (int temp = 0; temp < nList.getLength(); temp++)
      {
        Node nNode = nList.item(temp);
       System.out.println("\nCurrent Element :" +
       nNode.getNodeName()); if (nNode.getNodeType() ==
       Node.ELEMENT_NODE) {
          Element eElement = (Element) nNode;
          System.out.println("Student roll no : " + eElement.getAttribute("rollno"));
          System.out.println("First Name : " + eElement
            .getElementsByTagName("firstname").item(0).getTextContent());
          System.out.println("Last Name : " + eElement
            .getElementsByTagName("lastname").item(0).getTextContent());
          System.out.println("Nick Name : " + eElement
            .getElementsByTagName("nickname").item(0).getTextContent());
          System.out.println("Marks : "+ eElement.getElementsByTagName("marks")
            .item(0).getTextContent());
        }
      }
    }
```

```
catch (Exception e)
{
        e.printStackTrace();
    }
  }
}
```

This would produce the following result:

```
Root element :class
----------------------------

Current Element :student
Student roll no : 393
First Name : dinkar
Last Name : kad
Nick Name : dinkar
Marks : 85

Current Element :student
Student roll no : 493
First Name : Vaneet
Last Name : Gupta
Nick Name : vinni
Marks : 95

Current Element :student
Student roll no : 593
First Name : jasvir
Last Name : singn
Nick Name : jazz
Marks : 90
```

20. Explain in detail about SAX? **(11 MARKS)**

- The **Simple API for XML (SAX)** can only be used for parsing existing documents.
- SAX (Simple API for XML) is an event-driven online algorithm for parsing XML documents, with an API interface developed by the XML-DEV mailing list.
- SAX parsers operate on each piece of the XML document sequentially.
- SAX is an API that can be used to parse XML documents. A *parser* is a program that reads data a character at a time and returns manageable pieces of data.
- For example, a parser for the English language might break up a document into paragraphs, words, and punctuation.
- SAX provides a framework for defining event listeners, or ***handlers*.**
- These handlers are written by developers interested in parsing documents with a known structure. The handlers are registered with the SAX framework in order to receive events.
- Events can include start of document, start of element, end of element, and so on. The handlers contain a number of methods that will be called in response to these events. Once the handlers are defined and registered, an input source can be specified and parsing can begin.

SAX Basics

To illustrate how SAX works, consider a simple document:

```
<?xml version="1.0" encoding="UTF-8"?>
<fiction>
        <book author="Herman Melville"> Moby Dick
</book> </fiction>
```

If you want to parse this document using SAX, you would build a *content handler* by creating a Java class that implements the Content Handler interface in the org.xml.saxmpackage.

Once you have a content handler, you simply register it with a SAX XMLReader, set up the input source, and start the parser. Next, the methods in your content handler will be called when the parser encounters elements, text, and other data. Specifically, the events generated by the preceding example will look something like this:

```
start document
start element: fiction
start element: book (including attributes)
characters: Moby Dick
end element: book
end element: fiction
end document
```

The events reported follow the content of the document in a linear sequence. There are a number of other events that might be generated in response to processing instructions, errors, and comments.

XML Processing with SAX:

A parser that implements SAX (i.e., *a SAX Parser*) functions as a stream parser, with an event-driven API. The user defines a number of callback methods that will be called when events occur during parsing.

The SAX events include

- XML Text nodes
- XML Element Starts and Ends
- XML Processing Instructions
- XML Comments

SAX Packages

- The SAX 2.0 API is comprised of two standard packages and one extension package.
- The standard packages are
  - ✓ org.xml.sax
  - ✓ org.xml.helpers

org.xml.sax package

The org.xml.sax package contains the basic classes, interfaces, and exceptions needed for parsing documents.

Name	Description
Interfaces	
Attributes	Interface for a list of XML attributes.
ContentHandler	Receives notification of the logical content of a document.
DocumentHandler	ContentHandler interface, which includes namespace support.
DTDHandler	Receives notification of basic DTD-related events.
EntityResolver	Basic interface for resolving entities.
ErrorHandler	Basic interface for SAX error handlers.
Locator	Interface for associating a SAX event with a document location.
XMLFilter	Interface for an XML filter.
XMLReader	Interface for reading an XML document using callbacks.
Classes	
HandlerBase	DocumentHandler interface.
InputSource	A single input source for an XML entity.
Exceptions	
SAXException	Encapsulates a general SAX error or warning.
SAXNotRecognizedException	Exception class for an unrecognized identifier.
SAXNotSupportedException	Exception class for an unsupported operation.
SAXParseException	Encapsulates an XML parse error or warning.

(ii) org.xml.sax.helpers package

The org.xml.sax.helpers package contains additional classes that can simplify some of your coding and make it more portable

Class	Description
AttributesImpl	Default implementation of the Attributes interface.
DefaultHandler	Default base class for SAX2 event handlers.
LocatorImpl	Provides an optional convenience implementation of Locator.
NamespaceSupport	Encapsulate namespace logic for use by SAX drivers.
ParserAdapter	Adapts a SAX1 Parser as a SAX2 XMLReader.
XMLFilterImpl	Base class for deriving an XML filter.
XMLReaderAdapter	Adapts a SAX2 XMLReader as a SAX1 Parser.
XMLReaderFactory	Factory for creating an XML reader.

21. Explain how XML Integrating with database? (11 MARKS) (NOV 2016)

XML and database integration is important because XML provides a standard technique to describe data.

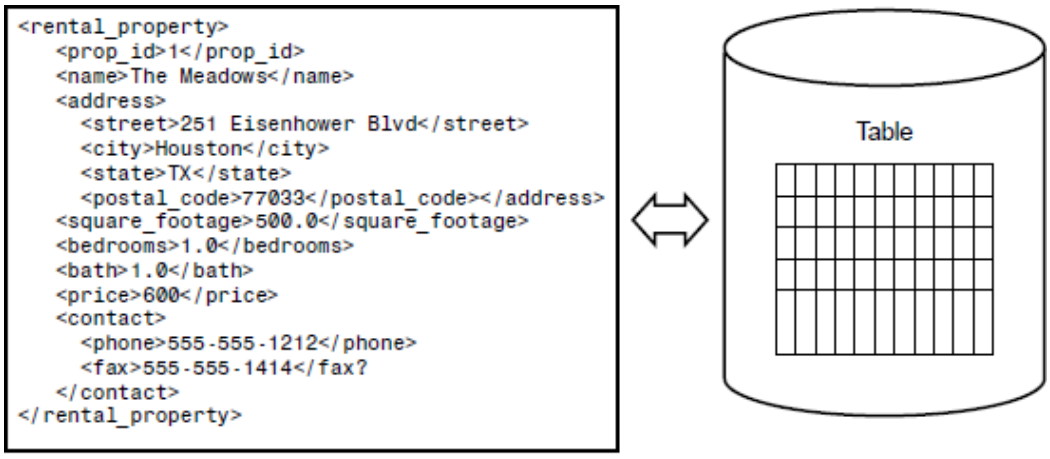
XML Database Solutions

A large number of XML database solutions are available, and they generally come in two flavors:

1. database mapping
2. native XML support

(i) XML Database Mapping

The XML database solution provides a mapping between the XML document and the database fields. The system dynamically converts SQL result sets to XML documents. Depending on the sophistication of the product, it may provide a graphical tool to map the database fields to the desired XML elements. Other tools support a configuration file that defines the mapping. These tools continue to store the information in relational database management system (RDBMS) format. The simply provide an XML conversion process that is normally implemented as a server-side Web application.



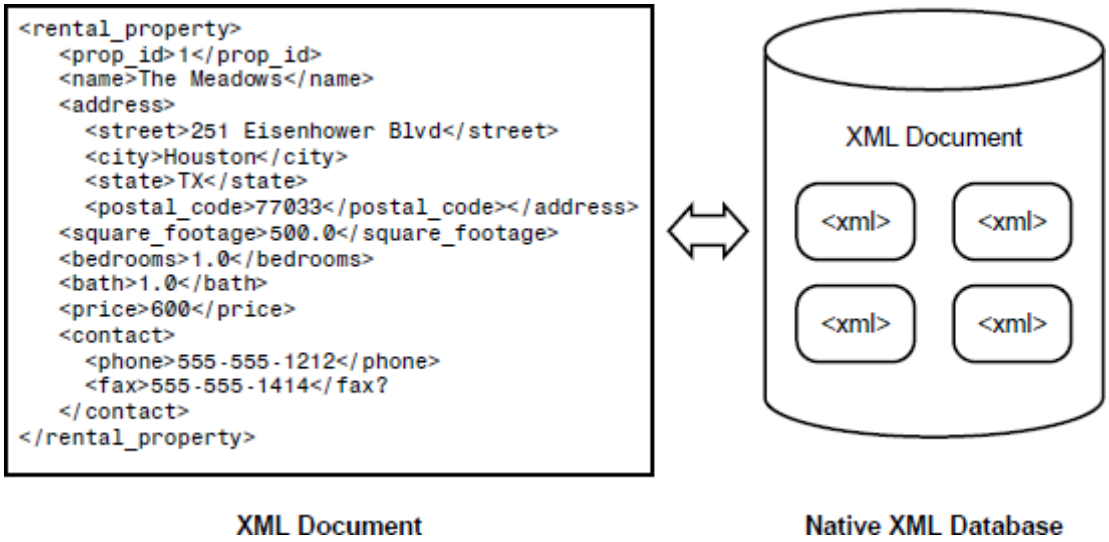
Mapping XML documents to database fields

XML Database Mapping Products

Product	Company	Web Site
DB2 Extender	IBM	www.ibm.com
SQL Server 2000	Microsoft	www.microsoft.com
Oracle 8i & 9i	Oracle	www.oracle.com
DataMirror DB/XML	DataMirror	www.datamirror.com
webMethods	webMethods	www.webmethods.com
Excelon	Excelon	www.exceloncorp.com

(ii) Native XML Support:

The XML database solution actually stores the XML data in the document in its native format. Each product uses its own proprietary serialization technique to store the data. However, when the data is retrieved, it represents an XML document.



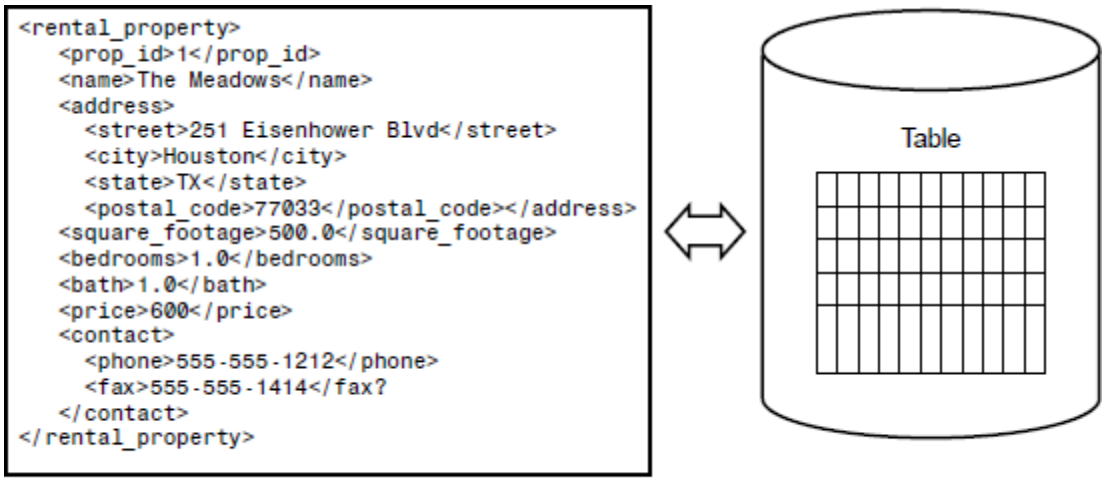
Native XML databases

Native XML Database Products

Product	Company	Web Site
TEXTM	L IXIA Soft	www.ixiasoft.com
Oracle 8i and 9i	Oracle	www.oracle.com
Excelon	Excelon	www.exceloncorp.com
dbXML	dbXML Group	www.dbxml.org
Tamino	Software AG	www.softwareag.com

Modeling Databases in XML:

Learn how to model a database in XML using Java. When we model a database, we provide an external representation of the database contents. For our sample program, we'll utilize a database that contains information on rental properties. We'll model the rental property database as an XML document.



One possible solution is to use Java servlets and JDBC. Java servlets are server-side components that reside in a Web server or application server. Java servlets are commonly used to handle requests from Web browsers using the HTTP protocol.

A key advantage to using servlets is the thin-client interface. The servlets handle the request on the server side and respond by generating an HTML page dynamically. This lowers the requirement on the client browser. The browser only has to provide support of HTML. As a result, there is zero client-side administration.

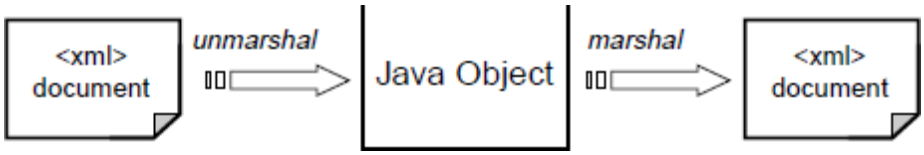
Java applets require the browser to support the correct version of the Java Virtual Machine (JVM). This has been a thorny issue with the Java community since the early days of applet development. If the browser doesn't support Java, the applet will not execute. Of course, there are a number of workarounds, such as the Java Plug-In and Java Web Start.

We can develop a servlet that uses JDBC. The servlet will make the appropriate query to the database and use Java database Connectivity (JDBC) API result set metadata to create the elements.

JAXB

The XML data binding features of **Java Architecture for XML Binding (JAXB)**. JAXB provides a framework for representing XML documents as Java objects. Using the JAXB framework, we can guarantee that the documents processed by our system are well formed. Also, we have the option of validating the XML data against a schema.

In the JAXB framework, we can parse XML documents into a suitable Java object. This technique is referred to as *unmarshaling*. The JAXB framework also provides the capability to generate XML documents from Java objects, which is referred to as *marshaling*.



JAXB marshaling and unmarshaling

JAXB is easier to use and a more efficient technique for processing XML documents than the SAX or DOM API. Using the SAX API, you have to create a custom content handler for each XML document structure. Also, during the development of the content, you have to create and manage your own state machine to keep track of your place in the document. Using JAXB, an application can parse an XML document by simply unmarshaling the data from an input stream.

JAXB is similar to DOM in that we can create XML documents programmatically and perform validation. However, the hindrance with DOM is the complex API. An XML tree, using the DOM API, we have to traverse through the tree to retrieve elements.

However, with JAXB, we retrieve the data from the XML document by simply calling a method on an object. JAXB allows us to define Java objects that map to XML documents, so we can easily retrieve data. The JAXB framework also ensures the type safety of the data.

JAXB Solution

1. Review the database schema.
2. Construct the desired XML document.
3. Define a schema for the XML document.
4. Create the JAXB binding schema.
5. Generate the JAXB classes based on the schema.
6. Develop a Data Access Object (DAO).
7. Develop a servlet for HTTP access.

22. Explain the formatting XML for the Web? **(11 MARKS)**

(i) XML Presentation Using CSS:

```
<!-- This style sheet will be referenced as notestyle.css -->
Note
{
    display: block
}
From, To
{
   display:block;
   font-family:verdana;
   font-size:15px;
   margin-bottom:5px
}
Subject
{
    display:block;
    font-family:verdana;
    font-size:13px;
    font-weight:bold;
    margin-bottom:10px
}
Body
{
    display:block;
    font-family:verdana;
    font-size:12px
}
```

In this listing are five style selectors: Note, From, To, Subject, and Body. Each selector listed represents the name of an XML element. The styles associated with each selector will be applied to XML elements that have matching names A CSS style sheet may be attached to an XML document through the use of the special XML processing instruction <?xml-stylesheet?>. There are two attributes to the xml-stylesheet processing instruction: type and href. The type attribute sets the MIME type for the CSS style sheet. Its value should always be text/css. The href attribute gives the URL for the location of the CSS style sheet.

The href attribute of the xml-stylesheet processing instruction assumes that the CSS style sheet is located in the same directory. The href attribute value could also be a relative URL or an absolute URL.

Applying CSS to an XML Document

```
<?xml version="1.0"?>
<!--
    This is referencing the style sheet from calling it notestyle.css here
-->
<?xml-stylesheet type="text/css" href="notestyle.css"?>
<Note>
    <From> From: Bob </From>
    <To> To: Jenny </To>
    <Subject> Subject: Hello Friend! </Subject>
    <Body> Just thought I would drop you a line.
</Body> </Note>
```

Output

From: Bob

To: Jenny

Subject: Hello Friend!

Just thought I would drop you a line.

(ii) XHTML

- XHTML stands for E**X**tensible **H**yper **T**ext **M**arkup **L**anguage
- XHTML is almost identical to HTML
- XHTML is stricter than HTML
- XHTML is HTML defined as an XML application
- XHTML is supported by all major browsers

Variants of XHTML

XHTML, a document must be validated against one of three DTDs

1. Strict
2. Transitional
3. Frameset

Strict DTD

A strictly conforming XHTML document that references the Strict DTD will have the following Document Type Declaration:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Strict DTD means that the XHTML document will have the following characteristics:

- There will be a strict separation of presentation from structure. Style sheets are used for formatting, and the XHTML markup is very clean and uncluttered. There are no optional vendor-specific HTML extensions.
- The Document Type Definition must be present and placed before the <html> element in the document.
- The root element of the document will be <html>.
- The <html> element will have the xmlns attribute in order to designate the XHTML namespace.
- The document is valid according to the rules defined in the Strict DTD.

Transitional DTD

The Transitional DTD for XHTML has more loosely defined requirements than the Strict DTD. As such, it is much easier to use with current Web browsers than the Strict DTD. To be more specific, you have to make far fewer changes to your existing Web pages. As long as the Transitional DTD is referenced from the Document Type Definition, the HTML is well formed, and it follows the basic XHTML syntax rules.

A Document Type Declaration containing a reference to the Transitional DTD will appear as follows:

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

Frameset DTD

The XHTML Frameset DTD is designed specifically to work with HTML frame pages. Frame pages are pages in which the browser has been broken up into several semi-independent navigable windows. Each frame, or window, will have its own content that is maintained in a file separate from the content in the other windows. Normally, one frame will contain navigation links and the other frame serves as the target for the link, loading whatever content the link points to when clicked. A frame page might be useful if you want to be able to load content from another Web site in one frame while keeping your navigation links available in another window. In addition to the files that make up the content for each of the frames, one main frame page "binds" the other frames together.

From this main page, you will reference the Frameset DTD. The Frameset DTD contains rules that apply specifically to the special setup of a frame page. In order to reference the XHTML Frameset DTD, use the following Document Type Declaration:

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">

Syntax for XHTML

- ✓ XHTML Must Be Well Formed
- ✓ All Elements and Attributes Must Be Lowercase
- ✓ Attribute Values Must Always Appear in Quotes
- ✓ Attributes May Not Be Minimized
- ✓ Script and Style Elements Must Be Enclosed in CDATA Sections
- ✓ Element Identifier References Are to the id Attribute

Supported XHTML Features

- Text Support
- Hyperlinks and Linking to Documents
- Table Support
- Forms Support
- Style Sheet Support
- Images Support

XHTML Basic Document

The XHTML Basic document type is composed of a set of XHTML modules.

Module	Contained Elements
Structure Module	body, head, html, title
Text Module	abbr, acronym, address, blockquote, br, cite, code, dfn, div, em, h1, h2, h3, h4, h5, h6, kbd, p, pre, q, samp, span, strong, var
Hypertext Module	A
List Module	dl, dt, dd, ol, ul, li
Basic Forms Module	form, input, label, select, option, textarea
Basic Tables Module	caption, table, td, th, tr
Image Module	img
Object Module	object, param
Metainformation Module	meta
Link Module	link
Base Module	base

XForms:

- XForms is the next generation of HTML forms
- XForms is richer and more flexible than HTML forms
- XForms is platform and device independent
- XForms separates data and logic from presentation
- XForms uses XML to define form data
- XForms stores and transports data in XML documents
- XForms contains features like calculations and validations of forms
- XForms reduces or eliminates the need for scripting

In XForms, there are separate sections used to describe the purpose of the form and the presentation of the form. This provides a great advantage over HTML because XForms makes no requirement of how the form should be presented. This makes XForms very rich and flexible. XForms may just as easily be displayed using XHTML form controls as WML form controls. This is because XForms does not predetermine the type of control that must be used to collect the data. XForms merely dictates the type of data that should be collected.

XForms Web Form

```
<xform:input ref=”txtName”>
<xform:caption> Name </xform:caption>
</xform:input>
<xform:selectOne ref=”cboAge”>
<xform:caption> Age </xform:caption>
<xform:choices>
<xform:item value=”Less than 18”> <xform:caption>Less than
18 </xform:caption> </xform:item>
<xform:item value=”18 - 35”> <xform:caption> 18 - 35 </xform:caption> </xform:item>
<xform:item value=”Over 35”> <xform:caption> Over 35 </xform:caption> </xform:item>
</xform:choices>
</xform:selectOne>
<xform:selectOne ref=”cboGender”>
<xform:caption> Gender </xform:caption>
<xform:choices>
<xform:item value=”Male”> <xform:caption> Male </xform:caption> </xform:item>
<xform:item value=”Female”> <xform:caption> Female </xform:caption> </xform:item>
</xform:choices>
</xform:selectOne>
<xform:submit>
<xform:caption> Submit <xform:caption>
</xform:submit>
```

Submitted XForms Data

```
<Envelope>
 <Body>
    <txtName> Michael Qualls </txtName>
    <cboAge> 18 – 34 </cboAge>
    <cboGender> Male </cboGender>
 </Body>
</Envelope>
```

XForms submits well-formed XML data. Once this data is received by the server, it may be validated against a DTD by an XML parser

Based on XML

XForms is, of course, an XML technology. This opens up XForms to using all the different permutations of XML on the Web, such as XHTML, WML, and XSL. It appears that XSL is going to be very important in relation to XForms. XSL will be able to be used to transform XForms documents to meet the presentation requirements of any clients using the form.

Platform Neutral

XForms is platform neutral. As an XML technology, no specific requirements are made in order to use XForms. XForms may be formatted for display as easily for handheld devices, as for cellular phones, and as for desktop computers.

XForms: Three Layers:

XForms forms technology is split into three layers. These layers are

1. Purpose layer
2. Presentation layer
3. Data layer

Purpose Layer

The purpose of the form and the presentation of the form are separated in XForms. The presentation was predetermined for any clients loading the page. The actual main form elements, referenced from the XForms namespace, are input and selectOne. The input element expects some input from the user, and selectOne allows the user to choose one of several defined options.

Presentation Layer

The presentation layer of XForms is actually dependent on the client loading the XForms document. An XForms document could be rendered in HTML, XHTML, or WML. XForms could even be configured for delivery to an audio device or a Braille device.

Data Layer

The data is entered into the form fields that are rendered by the client device. The data should be part of the presentation layer or appropriate to make data part of the purpose layer because the purpose layer defines the type of data being gathered.

```
<xform:xform>
    <xform:submitinfo action=”processexample.asp” method=”post”
    /> <xform:instance>
            <personalinfo >
            <txtName />
            <cboAge />
            <cboGender />
            </personalinfo>
    </xform:instance>
</xform:xform>
```

23. Create a XML document for students mark list preparation. Write DTD for validating the XML file. (11 MARKS) (APR 2016)

Create a DTD file for the XML file

- A DTD (Document Type Definition) is a set of rules that defines what tags appear in a XML document, how and where these tags appear, what the relationship the tags have with each other. Be brief, a DTD defines the legal elements and their structure in an XML document.
- When an XML document is processed, it is compared within the DTD to be sure the structure is valid and all tags are used in the proper manner. Independent developers can agree to use a common DTD for exchanging XML data, which makes data be shared easily.

students.dtd file

```
<?xml version = "1.0"?>
<!--students.dtd-a document type definition for the students.xml--
> <!ELEMENT students (student+)> <!ELEMENT student
(name,address)>
<!ELEMENT name (firstname,lastname)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT address (street,city,email,phone)>
<!ELEMENT street (#PCDATA)>
<!ELEMENT city (#PCDATA)>
```

```
<!ELEMENT email (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
```

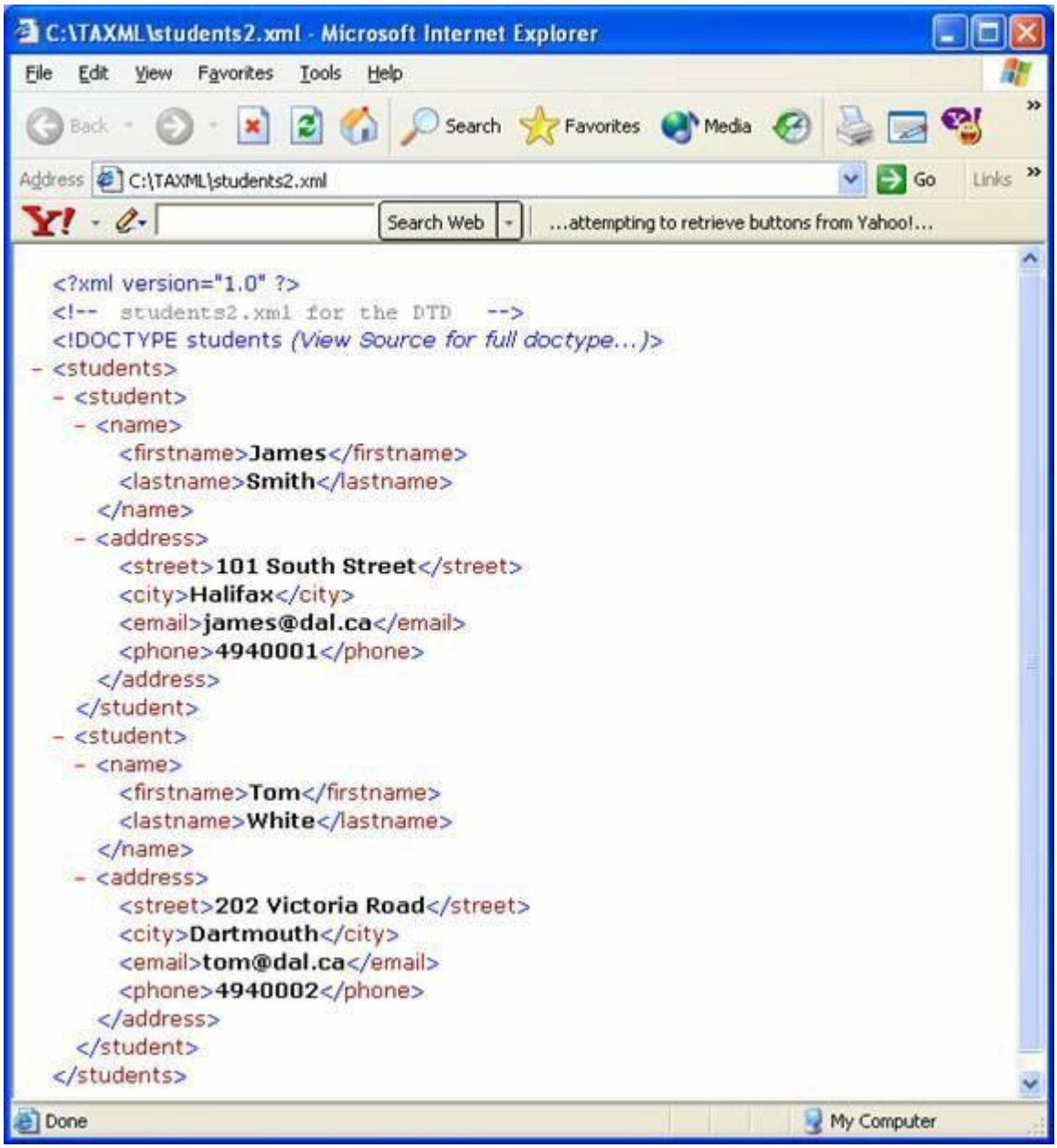
To use the DTD file, we must add this code into the XML file.

<!DOCTYPE students SYSTEM "**students.dtd**">

students2.xml

```
<?xml version = "1.0"?>
<!-- students2.xml for the DTD -->
<!DOCTYPE students SYSTEM "students.dtd">
<students>
    <student>
        <name>
            <firstname> James </firstname>
            <lastname> Smith </lastname>
        </name>
        <address>
            <street> 101 South Street</street>
            <city> Halifax </city>
            <email> james@dal.ca </email>
            <phone> 4940001 </phone>
        </address>
    </student>
    <student>
        <name>
            <firstname> Tom </firstname>
            <lastname> White </lastname>
        </name>
        <address>
            <street> 202 Victoria Road </street>
            <city> Dartmouth </city>
            <email> tom@dal.ca</email>
            <phone> 4940002 </phone>
        </address>
    </student>
<students>
```

Output:


C:\TAXML\students2.xml - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites Media
Address C:\TAXML\students2.xml
Go Links
Search Web
...attempting to retrieve buttons from Yahoo!...
<?xml version="1.0" ?>
<!-- students2.xml for the DTD -->
<!DOCTYPE students (View Source for full doctype...)>
- <students>
- <student>
- <name>
<firstname>James</firstname>
<lastname>Smith</lastname>
</name>
- <address>
<street>101 South Street</street>
<city>Halifax</city>
<email>james@dal.ca</email>
<phone>4940001</phone>
</address>
</student>
- <student>
- <name>
<firstname>Tom</firstname>
<lastname>White</lastname>
</name>
- <address>
<street>202 Victoria Road</street>
<city>Dartmouth</city>
<email>tom@dal.ca</email>
<phone>4940002</phone>
</address>
</student>
</students>
Done
My Computer

24. Illustrate the components of E-Business XML Systems. (11 MARKS) (APR 2016)

E-business systems aren't monolithic structures. They are comprised of major segments of functionality that help to contribute to an overall solution for supply chain interactions.

Because e-business in an Internet context, (especially using XML) is very new, not all components are necessarily implemented by all organizations.

However, a complete e-business system needs to be comprised of the following elements:

1) Enterprise and back-end integration
2) Various network and foundational layers
3) Messaging in a transport, routing, and packaging context
4) Registries and repositories
5) Data dictionaries
6) Process and workflow
7) Trading Partner Agreements
8) Business vocabularies

Enterprise Integration

Because e-business systems are a core part of any business enterprise that interacts with suppliers and trading partners, they cannot exist outside of and be disconnected from enterprise back-end systems. As a result, e-business systems need to be tied to various back-end systems, including the following:

- ✓ Customer Relationship Management (CRM) systems
- ✓ Enterprise Resource Planning (ERP) systems
- ✓ Asset- and inventory-tracking systems
- ✓ Point-of-sale systems
- ✓ Warehousing, shipping, and logistics systems
- ✓ Financial and accounting systems
- ✓ Marketing, sales, and customer service systems

Enterprise integration provides connection hooks into these various systems by means of APIs, Enterprise Application Integration (EAI), file transfer, or other shared messaging techniques. Integration is a two-way street, meaning that e-business systems can extract data from these various knowledge repositories as well as enter data into them. In the process of this bidirectional interchange, these systems apply business logic processing and translation among different formats. Enterprise integration therefore provides a gateway to the back-end systems from which results are communicated to other processes in the chain.

Fundamental Network and Platform Layers

These various layers cover the following fundamentals for e-business exchange:

- Partner connection and document transport
- Security
- Development platform and tools

e-business messages are generally transported over well-known and popular Internet transport protocols, including the Hypertext Transfer Protocol (HTTP), Simple Mail Transfer Protocol (SMTP), and File Transfer Protocol (FTP).

Additional security layers are applied to these fundamental transport protocols to provide security of different levels, including these:

- Encryption
- Authentication
- Authorization and permissions
- Privacy

Finally, the fundamental network components of e-business systems are the development tools and platforms for the construction of e-business systems, such as those powered by Web Services. Microsoft, Sun, HP, Oracle, and IBM all offer a number of e-business enabling systems that can be built on to offer the rich functionality required.

Messaging (Transport, Routing, and Packaging)

Messaging systems provide a standardized message and envelope structure that serves to identify endpoints in a given e-business transaction, specify how long messages are to be resent before timeout, and provide transaction controls and nonrepudiation (which helps to guarantee that messages are received by the end user). These components provide a certain level of session management and transaction coordination in a loosely coupled environment.

Messaging components also record sessions and other parameters that control reliable and secured messaging, among other features. As a result, messaging components serve as a basis for all e-business communications between parties.

Registry and Repository

E-Business services and capabilities are stored within repositories and registries—two terms whose meaning is often interchanged. Similar to the service offered by Universal Description, Discovery, and Integration (UDDI), registries and repositories serve as a central location where e-business services can be stored and later retrieved to dynamically discover business partners and their various capabilities, services, and business terms and conditions.

Data Dictionaries

Many portions of B2B information exchange require common knowledge of the vocabulary and acceptable items to be used within that vocabulary. These definitions of acceptable vocabulary usage can be found within a structure known as a data dictionary. These entities contain data structures, data types, constraints, and code lists of all the items necessary to compose valid business documents. In general, dictionaries specify the structure and semantics for particular business process documents.

Process and Workflow

Much of what differentiates e-business from simple e-commerce and individual transactional information is its ability to string multiple transactions into an overall business process to be executed. Many business process components, such as the "purchase order," are in fact composed of multiple individual transactions that must be executed in a particular order and with a given accepted workflow. In many e-business systems, these larger processes and workflows can be specified and exchanged in advance. Business processes can also be modeled with various technologies and those models shared to help craft the actual execution of e-business transactions.

Some business processes are applicable to a broad range of businesses, regardless of the vertical industry or locale, and despite specific characteristics of the business. These processes include many common business activities, such as invoicing, request for quote (RFQ), collaborative product development, purchasing, supply chain execution, and manufacturing. These general-purpose processes are defined so that they may be reused by other industries and businesses to achieve manageability and economies of scale. Other business processes are more specific to individual industries or organizations.

Trading Partner Agreements

In e-business, this electronic form is known as a Trading Partner Agreement (TPA). The TPA includes a profile of a business partner's contractual agreement for transaction as well as its e-business system infrastructure and usage of protocols.

Business Vocabulary

Business vocabularies have been defined by standards bodies of all sorts covering various business needs. These vocabularies are then used to construct the actual business content of a message. This message can contain product, finance, employee, or other business information.

Examples of business vocabularies include Health Level Seven (HL7) for the healthcare industry, ACORD for the insurance industry, the Open Travel Alliance (OTA) for the travel industry, and the Extensible Business Reporting Language (XBRL) for a variety of financial exchange needs.